Adversarial Search (AS):

- AS is a well suited approach in a competitive environment.
- AS is a technique in AI that helps agents to make decisions when competing against other agents.
- AS is used in games, busines strategy tools, and other competitive environments.

EB-SBIT

- AS works:
  - An AI Agent evaluates all possible moves and counter moves by the opponent.
- The agent aims to make optimal decisions while anticipating the opponent's action.
- The agent uses algorithms to evaluate and select the best moves.
- when agent goals are conflicts. give rise to adversarial search problems often called Games.
- zero sum games of perfect information

means when one player wins a game of s
(+1) and looser receives (-1)

- In games we include bridge, which includes elements of imperfect information b/c not all cards are visible to each player.

- optimal Decisions in games:
  - we will play game with two players max and min
  - max moves first and then they take turns moving until the game is over.
  - At the end points are awarded to the winning players and penalities are given to the looser.

- A game can be defined as a search problem with following components.
  - Initial state: which includes the board position and identifies the player to move.
  - succesor function: which returns a list of pairs, each indicates a legal move and the resulting state.

the value for the

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :

Year & Sem

Subject :

H.T. No. :

Date :

Signature of Invigilator

state (2)

- A terminal test: which determines when the game is over.
  states where the game has ended are called terminal states.
- A utility function (also called an objective function or payoff function) which gives numerical value to terminal states.
  - The outcome win, loose, draw +1, -1, 0
  - High value specifies Max (MAX)
  - low value specifies Min (MIN))

optimal strategies:

optimal solution is a sequence of moves leading to a goal state - a termal state is a win.

MINMAX Algorithm:

[Consider a simple example tic-tac-toe it is too complex to analyze]x

- Given a game tree, the optimal strategy can be determined by examining the minimax value of each node, we write it as MINIMAX-VALUE(n).

FR-SRIT

- The minimax value of a node is the utility (for MAX), assuming that both players play optimally from there starting to the end of the game.
- MAX will prefers val maximum value.
- MIN) prefers minimum value.

MINIMAX-VALUE(n) =

$$\begin{cases} \text{UTILITY}(n) & \text{if } n \text{ is a terminal state} \\ \max_{s \in \text{successors}(n)} \text{MINIMAX-VALUE}(s) & \text{if } n \text{ is a MAX node} \\ \min_{s \in \text{successors}(n)} \text{MINIMAX-VALUES}(s) & \text{if } n \text{ is a MIN node} \end{cases}$$

A two ply game-tree.

The △ nodes are MAX nodes.
- in which it is MAX's turn to move.
& ▽ nodes are MIN nodes.
- The terminal nodes show the utility values for MAX;
- The other nodes are labeled with their minimax values.
- Re terminal nodes on the bottom level are already labeled with their utility values.

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :

Year & Sem

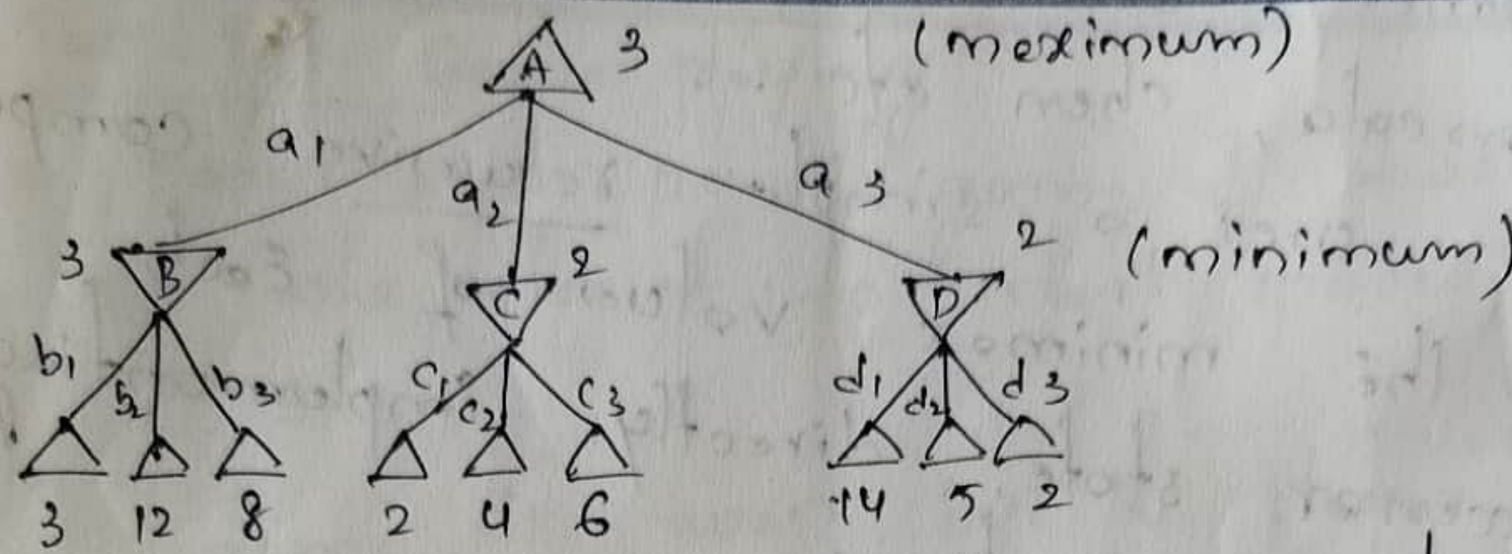
Subject : AI

H.T. No. :

Date :

Signature of Invigilator

(3)



f:g. A two ply game tree

- The bottom level values are the maximum values
- The first MIN node B has 3 successors (b1, b2, b3) from the of the MIN value is to be considered if is 3 minimax
- similarly the other two C and D
- The minimax values for C and D are 2, 2.
- The root node is the MAX node from the root A if has 3 successors (B, C, D) from this the highest minimax is 'B'
- MINMAX algorithm is a kind of backtracking algorithm i.e used in decision making and some theory to find optimal move

FB-SBIT

for a player, assuming that your opponent also plays optimally.

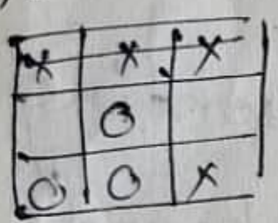
- It is used in two player turn-based games such as Tic-Tac-Toe, Backgammon, Mancala, chen etc..
- It uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations.
- The recursion proceeds all the way down to the leaves of tree,
- The minimax are backed up through the tree as the recursion unwinds.
- The minimax algorithm performs a complete depth-first exploration of the game tree.
- If 'm' is the maximum depth of the tree at each point.
- 'b' legal moves at
- Time complexity is $O(b^m)$
- Space complexity is $O(bm)$ for an algorithm that generates successors at once ~~or $O(m)$~~
- $O(m)$ that generates successor one at a time.

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

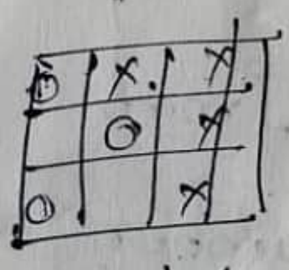
Name :
Year & Sem
Subject : AI
H.T. No. :
Date :
Signature of Invigilator

Tic-Tac-Toe Game playing

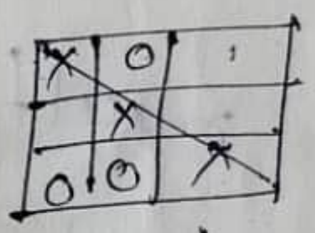
- It's a paper and pencil game of two player X and O, who choose to mark spaces on a grid of 3×3.
- The game is won by the player who succeeds in putting three of their marks in a horizontal, vertical or diagonal line



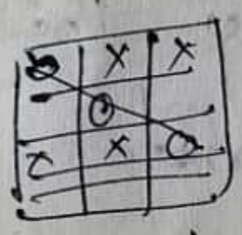
X wins



X wins

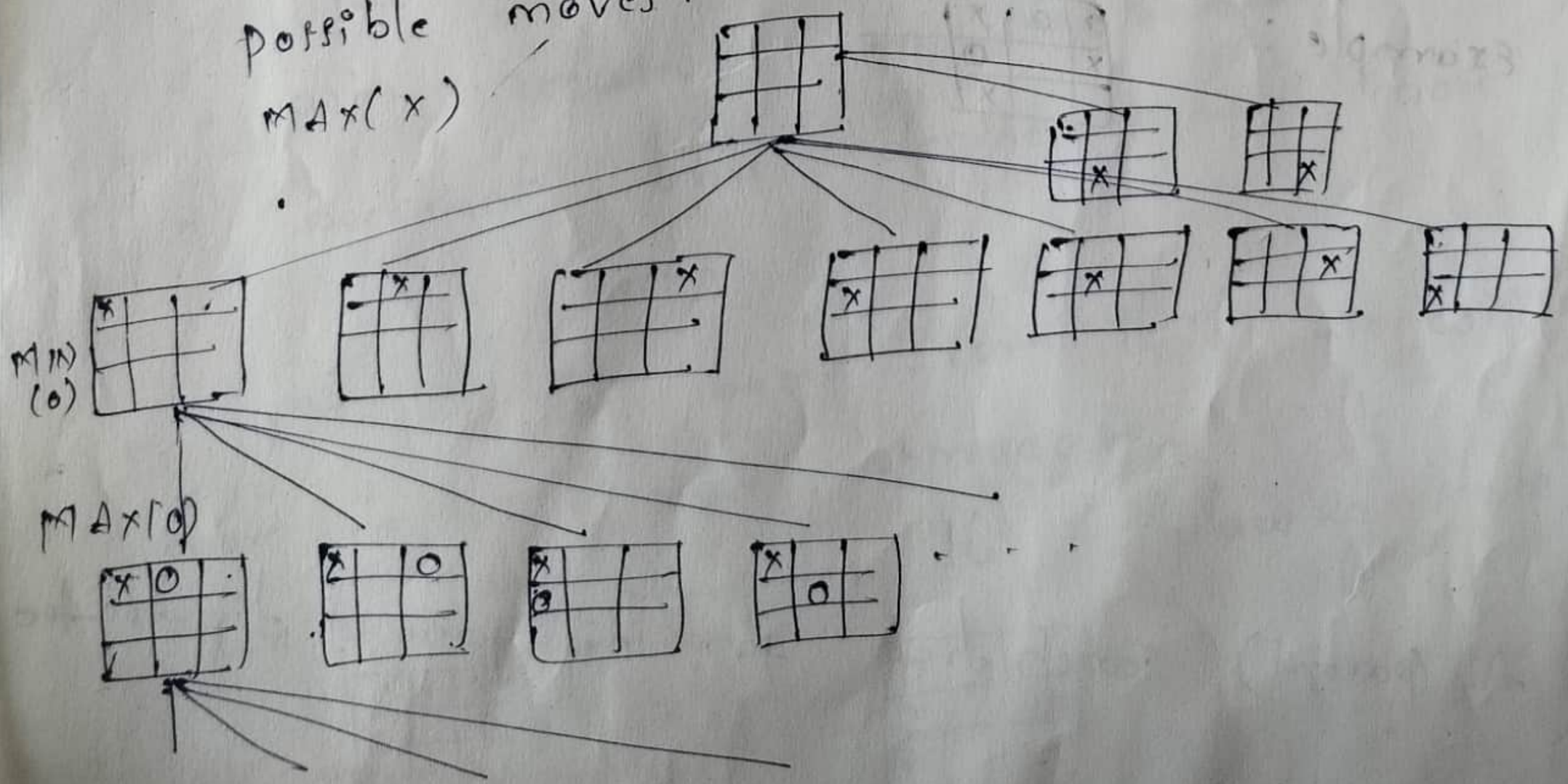


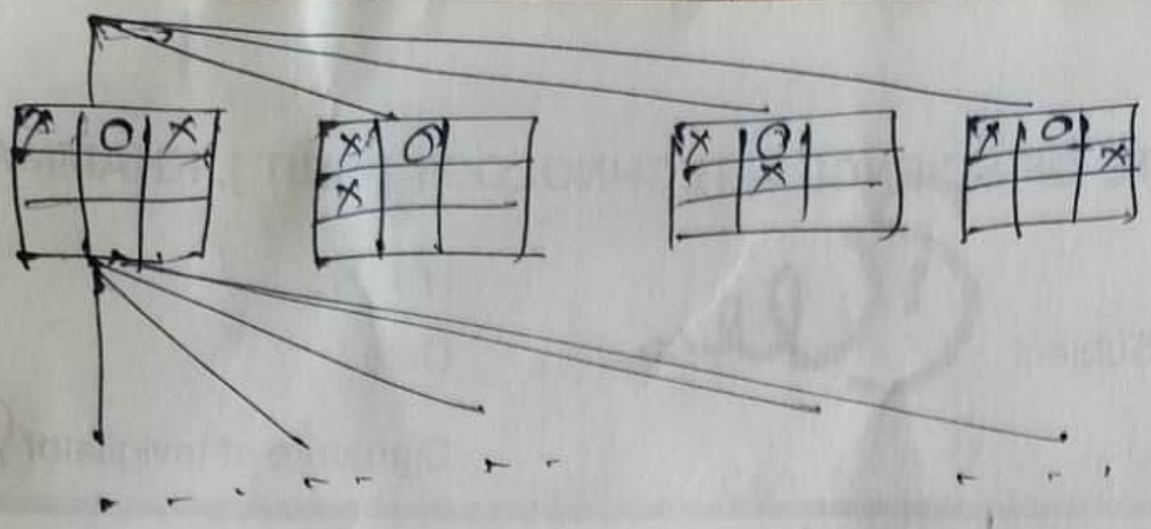
X wins.



O wins.

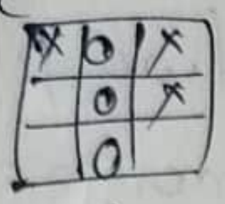
- From the initial state MAX has nine possible moves.



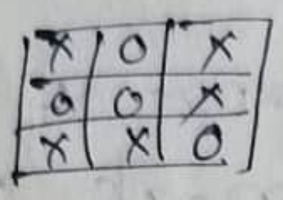


TERMINAL

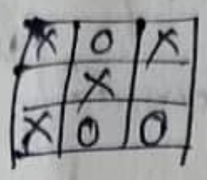
utility



-1



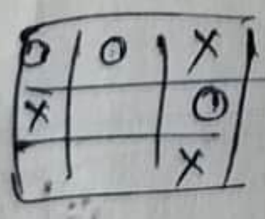
0



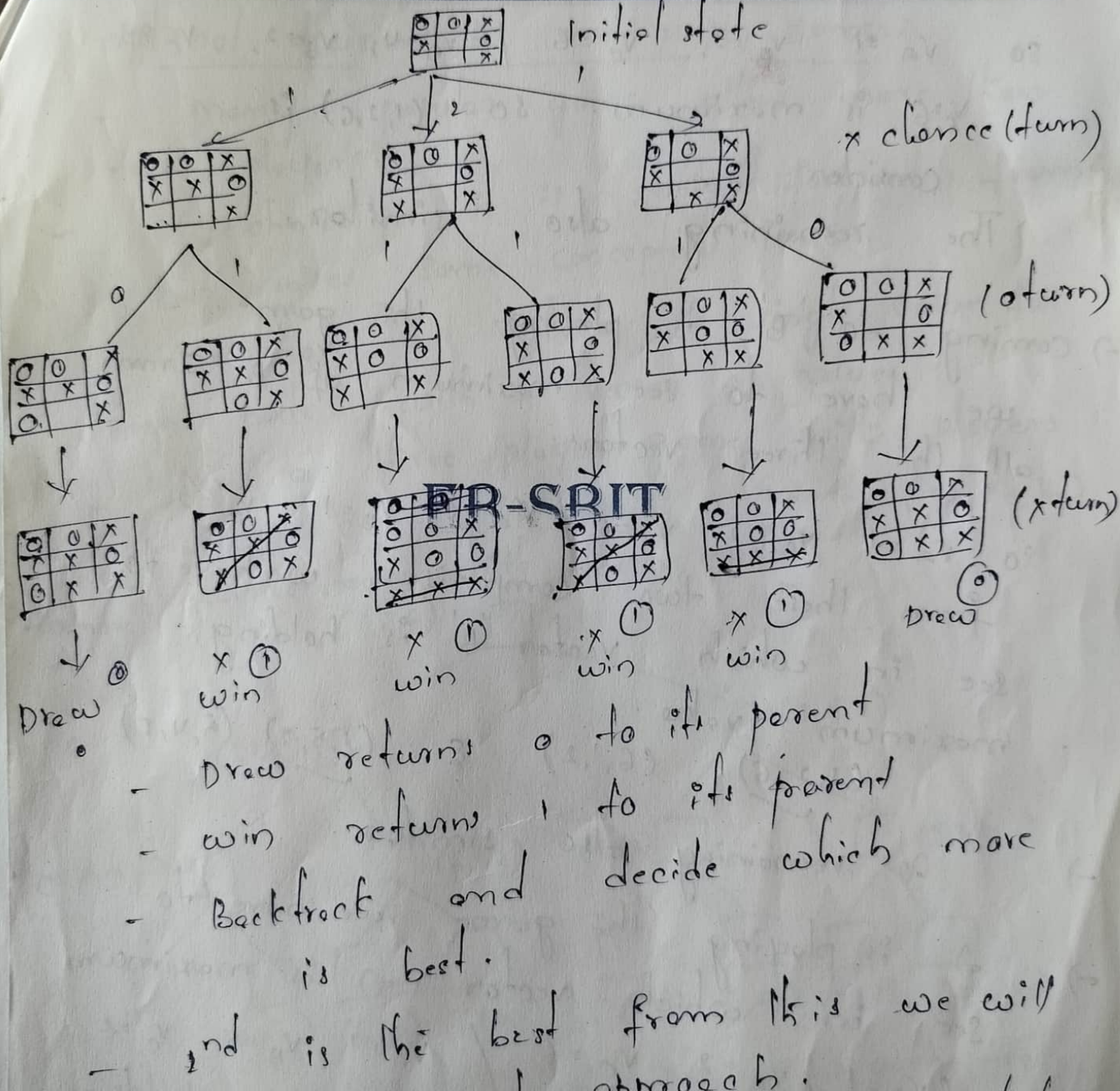
+1

- Data structures:
  - Board position is a structure containing
    - A 9 element array representing the board.
    - A list of board positions that could result from the next move
    - A number or rating representing an estimate of how likely the board position is to lead to an ultimate win for the player to move.

Example:



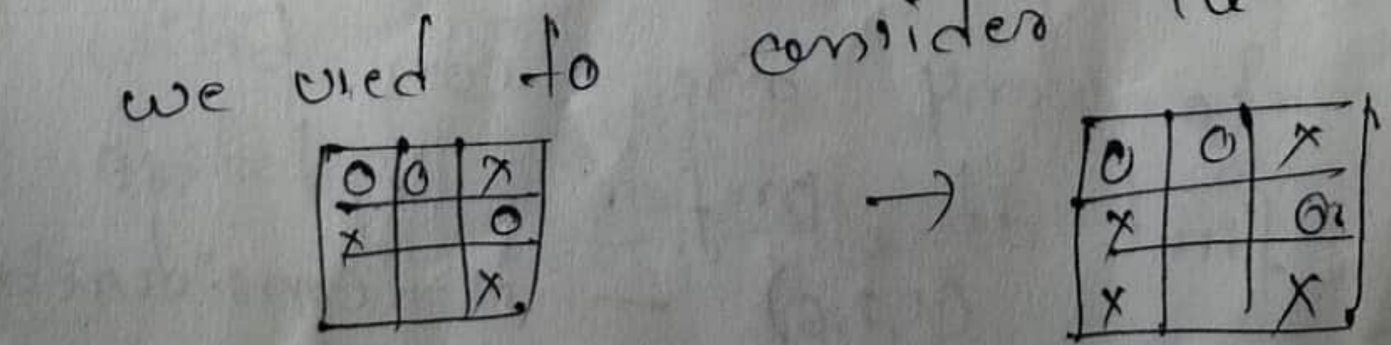
A (partial) search tree for the game of tic-tac-toe



FR-SBIT

- Draw returns 0 to its parent
- win returns 1 to its parent
- Backtrack and decide which move is best.
- 2nd is the best from this we will move to the next approach.

we used to consider the following state



next stage

*

In this level c is playing the game in the bottam level where the 'c' value is maximum i.e considered so $v_A = 1$, $v_B = 2$, $v_C = 6$ / $v_A = 4$, $v_B = 2$, $v_C = 3$ $v_C = 6$ is maximum. so (1, 2, 6) is consider.

The remaining also similar.

→ coming to 'B' is playing the game. we have to see maximum of B from all the three vectors.

so $v_A = 1$ $v_B = 2$ $v_C = 6$ / $v_A = 6$ $v_B = 1$ $v_C = 2$

from these two comparison. we have to see in which vector 'B' is holding maximum value.

(1, 2, 6) (6, 1, 2) & (1, 5, 2) (5, 4, 5)

→ remaining also similar.

→ A is playing the game we have to see in which vector A is maximum

($v_A = 1$, $v_B = 2$, $v_C = 6$ / $v_A = 1$ $v_B = 5$ $v_C = 2$

In this, in the both places A is holding same value so any one can be considered, first it prefer left so (1, 2, 6) — is considered.

optimal decisions in multiplayer games:

- many popular games allow more than two players.
- Technically it is straight forward from but raises some conceptual issues.
- we have to replace the single value for each node with a vector of values.
- In a three player-game with players A, B, and C a vector (v_A, v_B, v_C) is associated with each node.

FR-SRIT

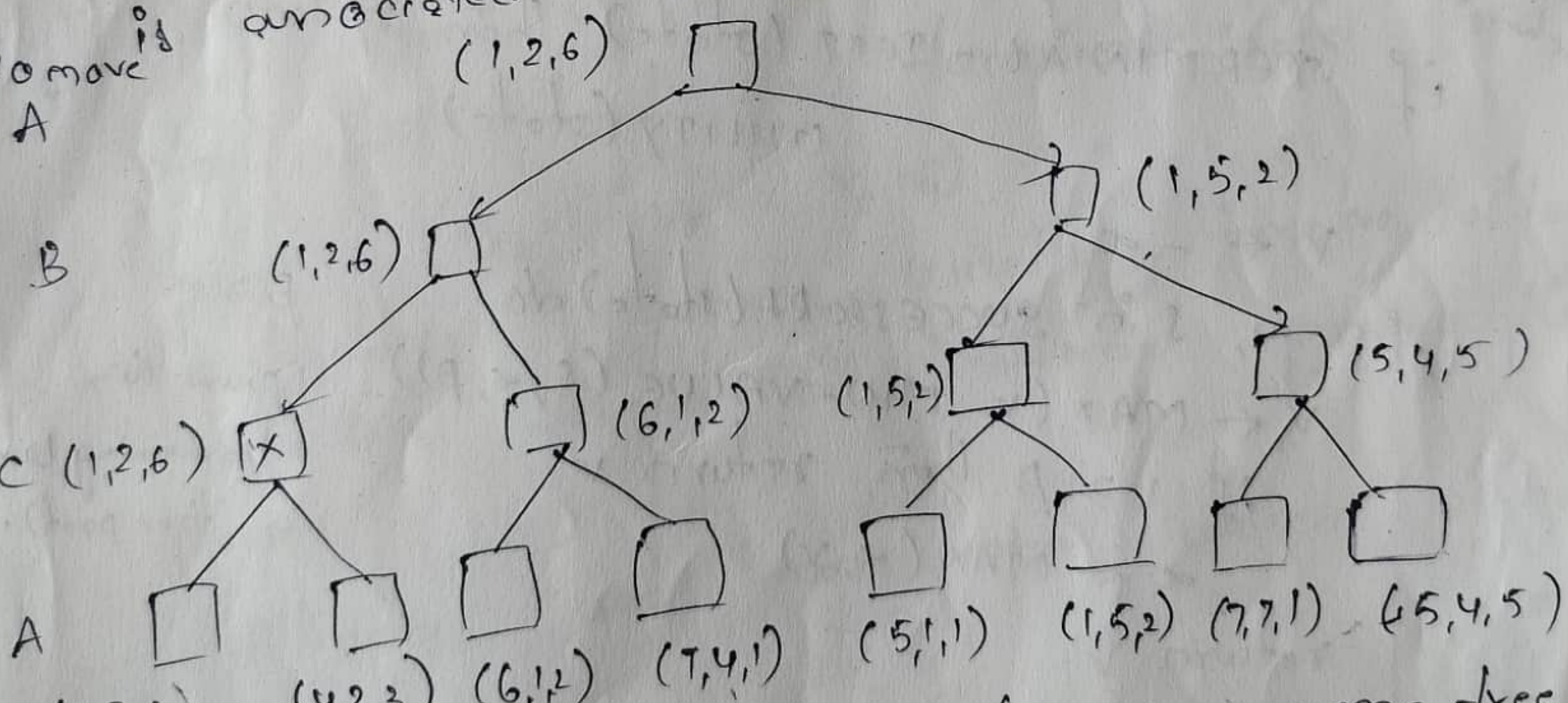


fig: The first three ply of a game tree with three players (A,B,C). Each node is labeled with values from the viewpoint of each player.

function ALPHA-BETA-SEARCH ~~MINIMAX~~ ~~DECISION~~ (state) returns an action,

inputs: state, current state in game

$v \leftarrow$ MAX-VALUE (state, $-\infty$, $+\infty$)

return the action in SUCCESSORS (state) with value v

function MAX-VALUE (state, α, β) returns a utility value

inputs: state, current state in game.

α, the value of the best alternative for MAX along the path to state

β, the value of the best alternative for MIN along the path to state

($\alpha \geq \beta$) the remaining prune the remaining branches tree post.

if TERMINAL-TEST (state) then return UTILITY (state)

$v \leftarrow -\infty$

for a, s in SUCCESSORS (state) do $\beta \geq \alpha$

$v \leftarrow$ MAX (v, MIN-VALUE (s, α, β)) (prune the remaining branches tree post).

if $v \geq \beta$ then return v

$\alpha \leftarrow$ MAX (α, v)

return v.

$v \leftarrow +\infty$

$v \leftarrow$ MIN (v, MAX-VALUE (s, α, β))

if $v \leq \alpha$ then return v

$\beta \leftarrow$ MIN (β, v)

state.

- The utility function return a vector of utilities.
- In the above diagram the node marked x in the game tree, i.e player C chooses what to do.
- The 2 choices leads to utility vectors ($v_A = 1$, $v_B = 2$, $v_C = 6$) and ($v_A = 4$, $v_B = 2$, $v_C = 3$) since 6 is bigger than 3, 'c' should choose the first move.
- If state 'x' is reached the subsequent play will lead to terminal state with utilities ($v_A = 1$, $v_B = 2$, $v_C = 6$)
- multiplayer games usually involve alliances whether formal or informal among the player's.
- Alliances are made and broken as game proceeds.
- example if vector A and vector B are weaker in position vector C is stronger in position

FR-SBIT

- If it is optimal for A and B, they can attack 'c'
- Even vector 'c' destroy A and B individually.
- In this way collaboration emerges based on purely selfishness.
- The collaboration b/w two players also
- If the utility terminal state $V_A = 1000$, $V_B = 1000$, Then the optimal strategy is for both the players, then they will have mutual understanding, they will reach the goal.

so ..

- In this mostly we will follow the DFS strategy.

Alpha - Beta pruning:

The problem with minimax search is that the no. of game states it has to examine is exponential in the no. of moves.

- We can't eliminate the exponent, but we can cut into half.
- It is possible to compute the correct minimax decision without looking at every node in the game tree.
- In order to eliminate large parts of the tree from consideration we use the idea of pruning.
- This technique particularly called as alpha-beta pruning.

α - the value of the best (i.e highest-value) choice we have found so far at any choice point along the path for MAX

β - the value of the best (i.e lowest-value) choice we have found so far at any choice point along the path for MIN

FR-SRIT

pruning - eliminates the large parts of the tree.

- In this also it follows DFS, so at any one time we have to consider the nodes along a single path in the tree.

- The alpha-beta pruning will also follow the some policy as minmax, but it will prune the nodes that is not in use and won't disturb the decision tree

- Alpha (α) - Beta (β) proposes to find the optimal path without looking at every node in the game tree.

- Max contains 'α' and Min contains β during calculation.

- In both MIN and MAX nodes we return when $\alpha \geq \beta$ which compares with its parent node only.

- Both minimax and Alpha (α) - Beta (β) cut-off give some path.

- Alpha (α) - Beta (β) gives optimal solution as it takes less time to get the value for the root node.

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem
Subject : AI
H.T. No. :
Date :
Signature of Invigilator

- Adversarial search in AI involves finding optimal strategies in competitive, multiagent scenarios like games, where opponents have conflicting goals.
It uses algorithms called minimax and alpha-beta pruning to explore a game tree of possible moves.

FB-SBIT

key concepts.

- opposing Goals. Agents have conflicting objectives, aiming to win or achieve maximum utility.
- zero sum game. one player gain directly leads to another player's loss
- Game tree: it represents all possible moves of states from given position.

- Min-Max algorithm is claimed to be a recursive or backtracking algorithm

that is responsible for choosing the best optimal move in the conflicting environment.

- Min-max algorithm operates on a tree structure known as the game tree.

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem
Subject : AI
H.T. No. :
Date :
Signature of Invigilator

2.3

CSP is a mathematical problem where we confuse variable, Domains, constraints
CSP involves variables that are discrete and have finite domains.

- The maximum domain size of any variable in a csp is d, then the no. of possible complete assignments is $O(d^m)$.
- 'm' is the number of variables.
- finite domain csp's include Boolean csp's also which holds true and false.

FB-SBIT

- linear constraints are constraints that involve simple equations or inequalities b/w variables.
- These constraints can be easily handled using techniques like linear programming.
- non linear constraints, unlike binary or higher order constraints deal with relationship between more than two variables or complex combinations of them.

---

csp consists of 3 variables V, D, C
V is the set of variables represented as
$\{V_1, V_2, \ldots V_n\}$
D is the set of Domains represented as
$\{D_1, D_2 \ldots D_n\}$
- C is the set of constraints

- Unary constraint: In this constraint only one variable going to be used
  SA ≠ NSW ?
- Binary constraint: More than two variables can be used Ex: SA ≠ NSW.
- Higher order constraint: constraint considers 3 or more variables.
  ex: cryptoarithmetic puzzles.
  Each letter in cryptoarithmetic represent different digit.

```
  TWO
+ TWO
------
 FOUR
```

This would be represented as six variable constraint

allDif(F, T, U, W, R0)

The addition constraints on four columns of the puzzle can also involve several variable can be written as

$O + O = R + 10 \cdot X_1$

$X_1 + W + W = U + 10 \cdot X_2$

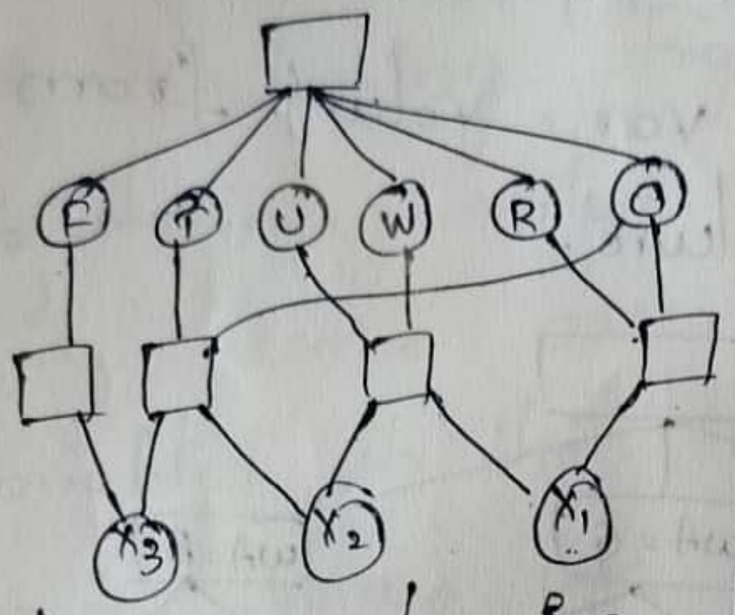
$X_2 + T + T = O + 10 \cdot X_3$

$X_3 = F$

X_1, X_2, X_3 are auxillary variables representing digits 0 or 1 carried over to next column.

higher-order constraints are represented in a constraint hypergraph.

Ex:-



constraint hypergraph for the cryptoarithmetic problem

FR-SRIT

- Backtracking search for CSPs:

function BACKTRACKING-SEARCH (csp) returns a solution or failure

return RECURSIVE-BACKTRACKING ({}, csp)

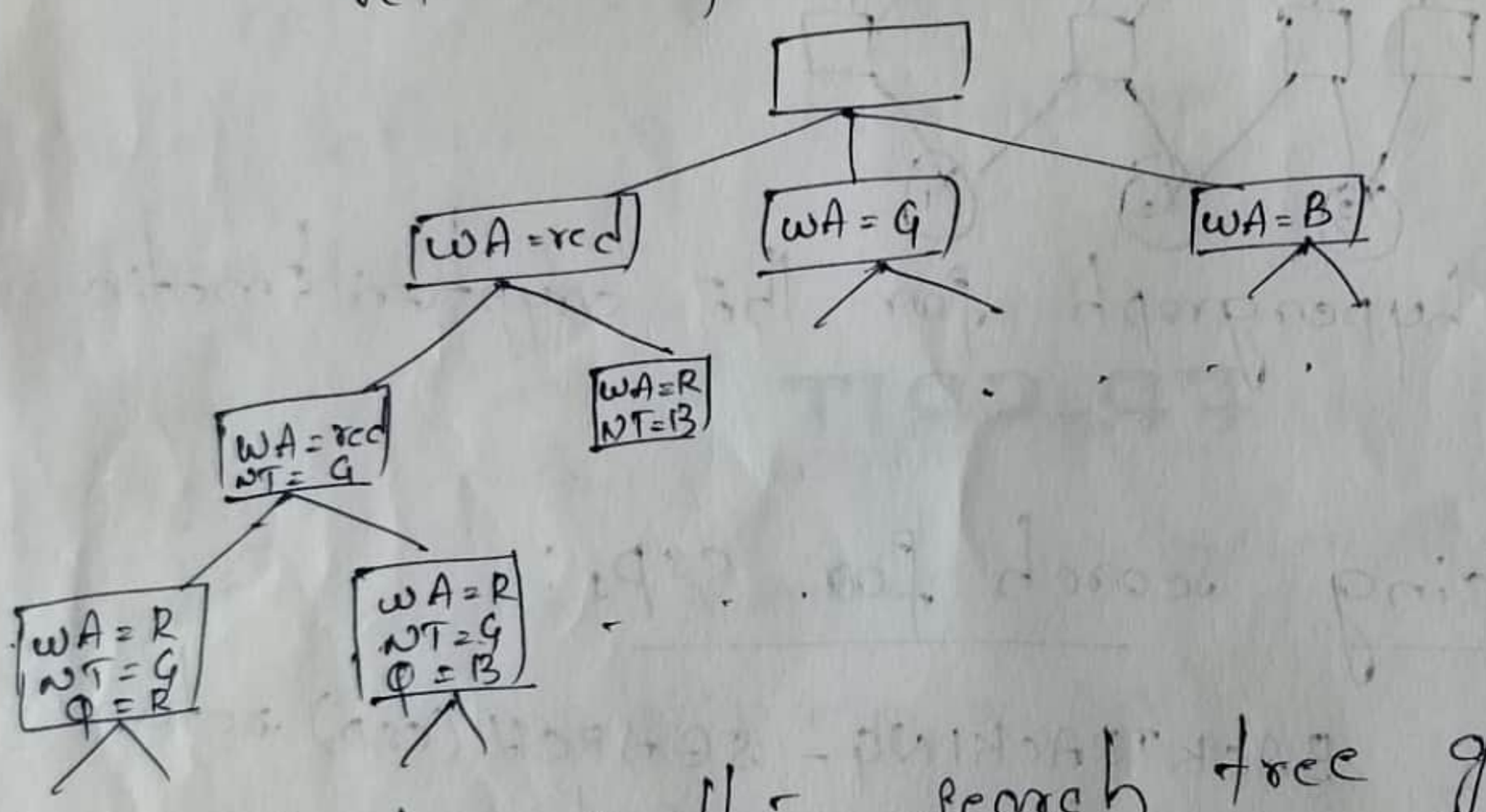
function RECURSIVE-BACKTRACKING (assignment, csp) returns a solution or failure

if assignment is complete then return assignment

var ← SELECT-UNASSIGNED-VARIABLE (VARIABLES[csp], assignment, csp)

for each value in ORDER-DOMAIN-VALUES (var, assignment, csp) do

if value is consistent with assig according to CONSTRAINTS [csp] then
add {var = value} to assignment
result ← RECURSIVE-BACKTRACKING (assignment, csp)
if result ≠ failure then return result
remove {var = value} from assignment
return failure.



part of the search tree generated by simple backtracking for the map coloring problem.

- Variable and value ordering:

The variables are given the value ordering of the domains, based on the variables placed in the list.

- choosing the variables with fewest legal values are called minimum remaining values (MRV).

- The degree heuristic is the variable that involved in many constraints.

Ex: SA is having highest degree.

- least constraining-value heuristic can be effective in some cases.
- The variables in the list along to that the domain values are assigned

WA given 1st NT given 2nd

Q given 3rd.

FR-SBIT

propegating information through constraints:

constraints on variables can only be used when SELECT-UNASSIGNED-VARIABLE can be used.

- one way to make better use of constraints during search is called forward checking

	WA			NT			Q			NSW			V			SA			T		
	R	G	B	R	G	B	R	G	B	R	G	B	R	G	B	R	G	B	R	G	B
Initial domains																					
WA=red	(R)				G	B	R	G	B	R	G	B	R	G	B		G	B	R	G	B
Q=green	(R)					B		(G)		R		B	R	G	B			B	R	G	B
V=blue	(R)					B		(G)		R				(B)					R	G	B

constraint propogation:

- Forward checking detects many inconsistencies, it cannot detect all of them.

Time, Item:

constraint propagation is a method that uses logical deductions to reduce the search space.

Backtracking: It is a method that uses recursion to try for different assignments of values to variables.

local search: It is a method that starts with an initial assignment and repeatedly (iteretively) applies small changes to find a better solution.

variables	WA	NT	SA	Q	NSW	V	T
initial domain	RGB	RGB	RGB	RGB	RGB	RGB	RGB
WA = R	R	GB	GB	GB	GB	GB	RGB
NT = G	R	G	B	RB			

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :

Year & Sem

Subject :

H.T. No. :

Date :

Signature of Invigilator

- csp example

  TWO
+ TWO

 FOUR

first this example represents on 6 variables constreints that if can be represented as Alldiff (F, T, U, W, R, O)

- This example can be written in the form of binary constreints i.e $F \neq T$ and $F \neq W$ by considering auxillary variables.
- we consider the constreints with all the different variables.
- It can also be represented as the binary constreints $F \neq U$, $F \neq R$, $F \neq O$.
- The constreint representation can be done by considering other variables can be written as

→ Performing addition we will from right side i.e variable $X_3 = F$.

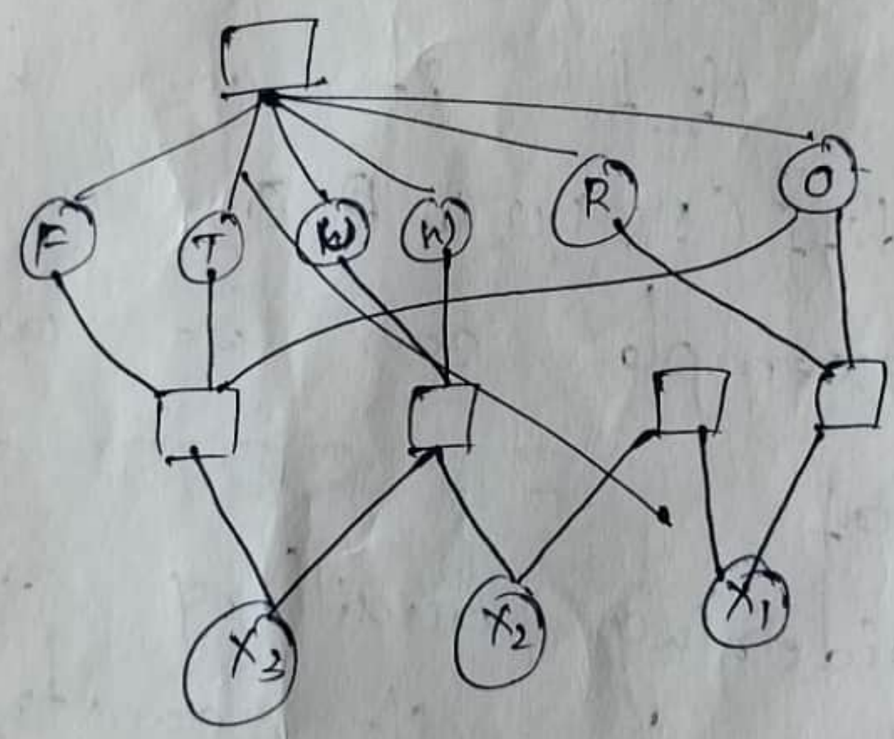
$$O + O = R + 10 * X_1$$

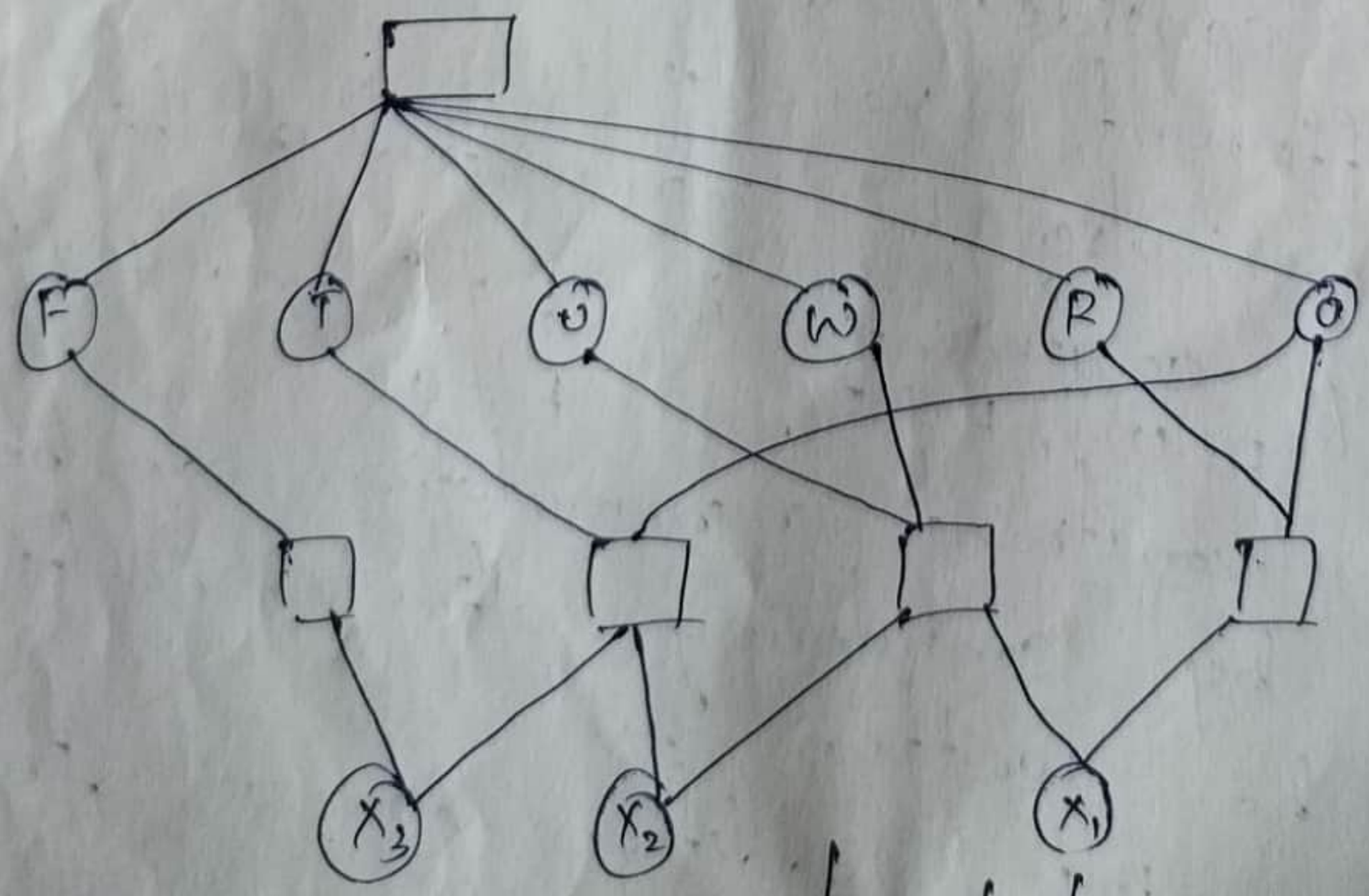
$$X_1 + W + W = U + 10 * X_2$$

$$X_2 + T + T = O + 10 * X_3$$

FR-SBIT

- f is the decoder of the domain
- Here x_1, x_2, x_3 are the auxillary variab representing the digit (0 or 1) carried over into nex column.
- It is a four column constraint.
- This relates to higher order constraints can be represented in a constraint hypergraph as below





constraint hypergraph.
for cyrptarithmetic

SWARNA BHARATHI INSTITUTE
Name:
Year & Sem
Constraint
node

Constraint propagation:

node consistency - unary constraint only one variable is considered.

SA = { Red, Green, Blue }

SA is a state if relates to 3 colors Domain

unary means SA ≠ G SA = { R, ~~R~~, B G

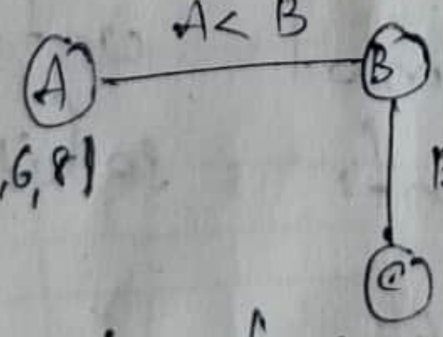
Arc consistency - binary constraint

every value in a variable's domain satisfies the variable binary constraint.

AC-3 algorithm.

it is called Arc 3 algorithm.

eg:-



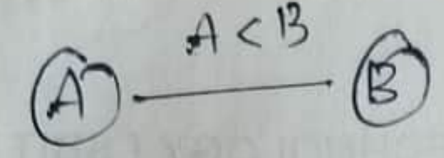
In this undirected graph A → B is an arc B → A is an arc B → C is an arc C → B is an arc

A-B	B-A	B-C	C-B

- these are arc's of the given undirected graph.

considering A-B arc.

- A-B is an arc. considering A-B first verifying A-B arc, if constraint A < B



{2,4,6,8}

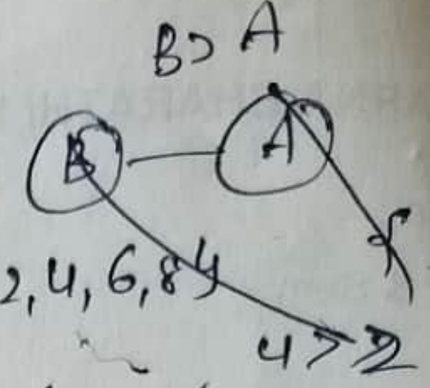
{2,4,6,8}

considering 2 < 4,6,8

4 < 6,8

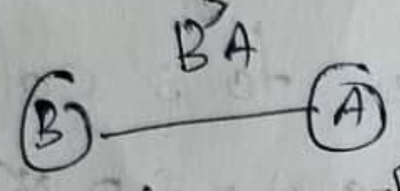
6 < 8

8 < ? no variable.



so remaining 8 from the A

So we have to consider the remaining



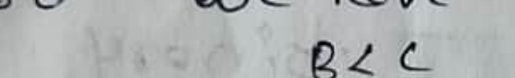
{2,4,6,8} {2,4,6}

4>2

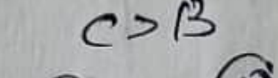
6>2, 6>4

8>2, 8>4, 8>6

So we have to remove 2.

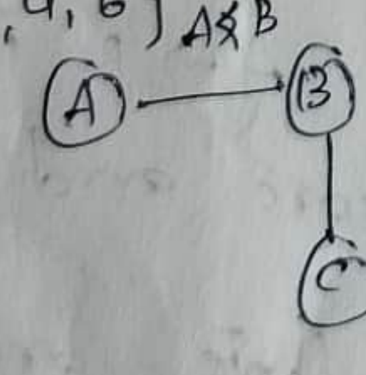
similly 

{4,6,8} {2,4,6,8}

and 

{2,4,6,8} {4,6}

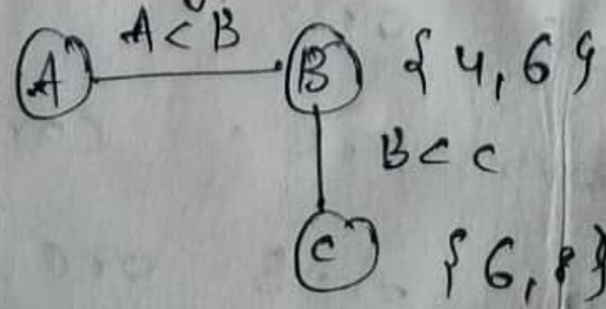
again we have to repeat the process in a queue format until the sequence is constraints satisfies

{2,4,6} 

{4,6}

{6,8}

continue same as previous we will get finally.

{2,4} 

{4,6}

{6,8}

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem

Subject : AI

H.T. No. :
Date :

Signature of Invigilator

Path consistency: Arc consistency can be used in this instead of 2 we are using 3 variables i.e

$wA \to sA$ if can be $wA \to NT \to SA$

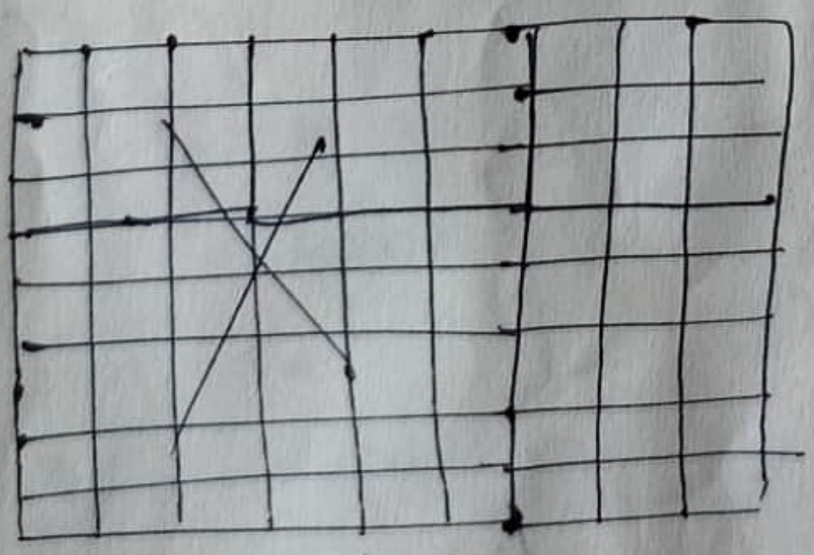
using bridge state it can be displayed.

K consistency: mean generalised form. specifying the number of variables. user used to specify the consistency based on the no. of variables been using.

FR-SRIT

Global constraints:

sudoku example

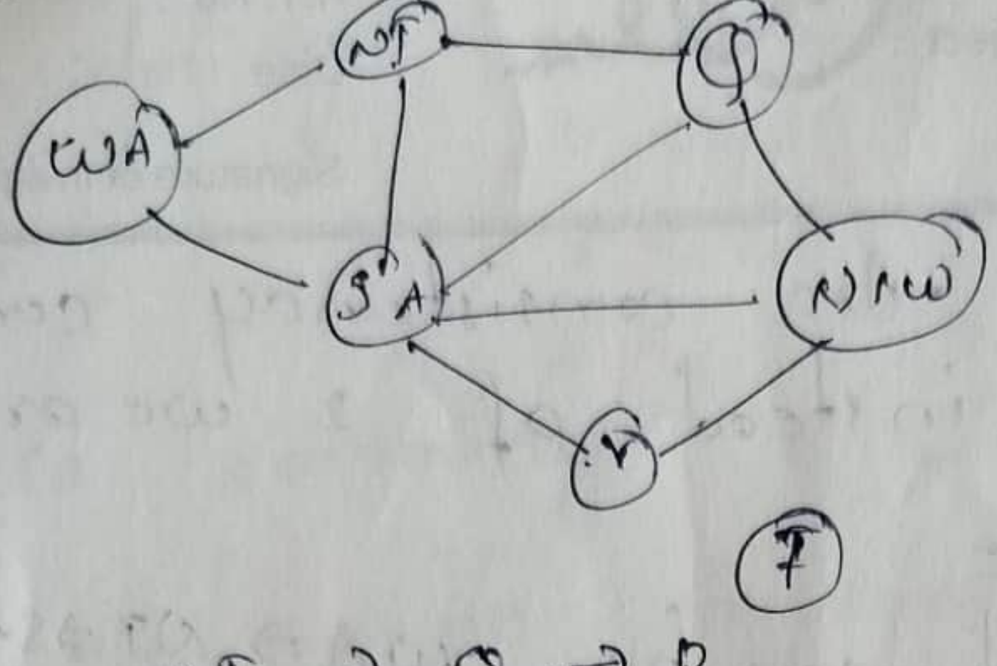


- value ordering:

Based on the constraint you are giving ordering value to the state.

- Even we have to depend to on least constraining value.

- we should not make any variab
domain empty.



(7)

WA — NT → Q → R
P Q B
SA becoming P Q

Minimum remaining values can also be called as fail-first heuristic or most constrained variable.

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :

Year & Sem

Subject : AI

H.T. No. :

Date :

Signature of Invigilator

Algorithm Arc consistency:

function AC-3 (csp) returns false if an inconsistency is found and true otherwise
{possible with reduced domains}
inputs: csp, a binary csp with components (X, D, C)
variables {$X_1, \ldots X_n$}
local variables: queue, a queue of arcs,
initially all arcs in csp
while queue is not empty do
(X_i, X_j) ← REMOVE-FIRST (queue)

EB-SBIT

for each X_k in NEIGHBORS [X_j] do
add (X_k, X_i) to queue

function REMOVE-INCONSISTENT-VALUES (X_i, X_j)
returns true iff we remove a value
removed ← false
for each x in DOMAIN [X_i] do
if no value y in DOMAIN [X_j] allows
(x, y) to satisfy the constraint b/w
X_i and X_j
then delete x from DOMAIN [X_i];
removed ← true
return removed.

The arc consistency Algorithm AC-3
- After applying AC-3 either every arc is arc consistent, or some variables has an empty domain, indicating that the csp can't be made arc consistent
- This csp cannot be solved
- Arc Algorithm invented Mackworth 1977.

local search Algorithm:

1. Initialization: start with an initial solution, which can be generated randomly or using a heuristic.
2. Neighborhood generation:
Generate a set of neighboring solutions by applying small changes to the current solution.
3. Evaluation: evaluate the objective function and constraints for each neighboring solution.
4. selection: select the best neighboring solution that satisfies the constraints and improves the objective function value.

5. update : update the current solution to the selected neighboring solution.

6. Termination . Repeat step 2-5 until a termination condition is met, maximum number of iterations or a satisfactory solution.

Advantages of local search csps are

① efficient.
② flexible
③ Robust

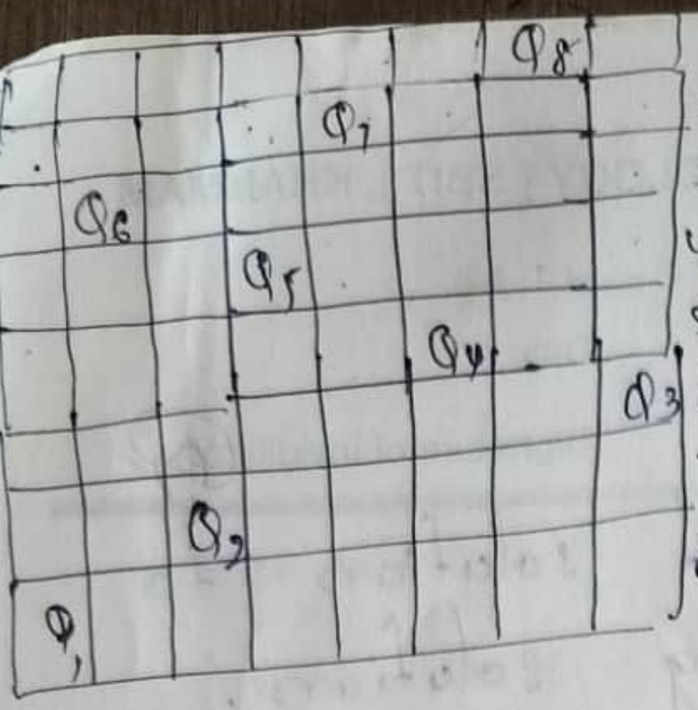
FR-SRIT

Techniques for continuous csps are.

1. Gradient - based methods (uses gradient
2. # information to guide the search algorithm.

2. Hill climbing : uses a simple iterative approach to improve the solution (increasing order).

3. Simulated annealing : uses a probabilistic approach to avoid getting stuck in local optima.



initial state
$q_1 = 8$
$q_2 = 7$
$q_3 = 6$
$q_4 = 5$

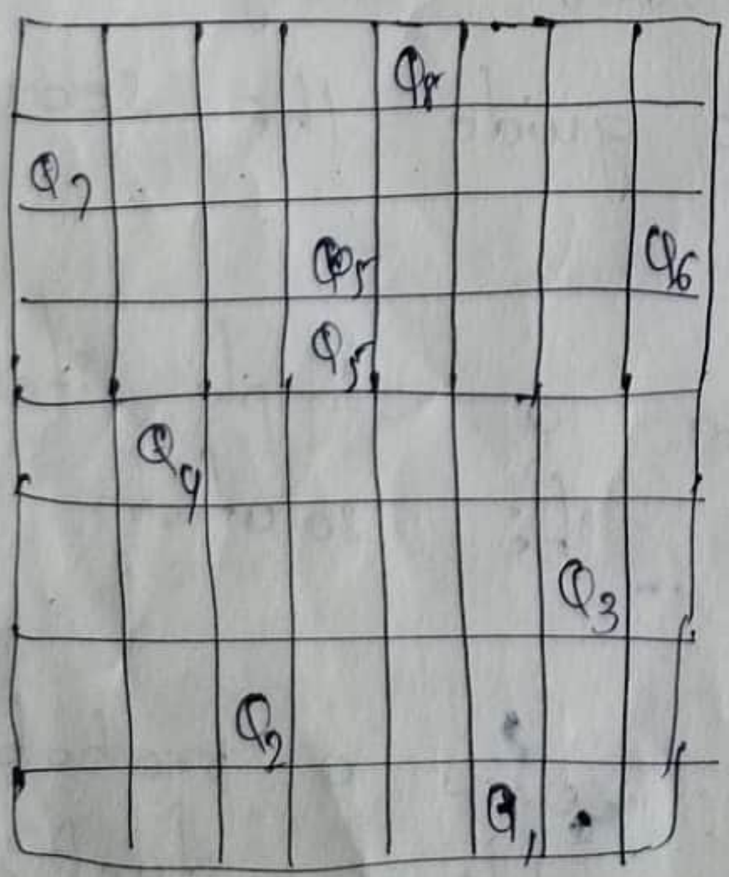
- uses a complete state formulation: the initial state assigns a value to every variable and the search changes the value of one variable at a time.

q_5 and q_8 are getting conflict so we have to change the place.

MIN_CONFLICT (csp, max_steps)

- in backtracking we start with 'a' anignment
- in local search we start with complete anignment.

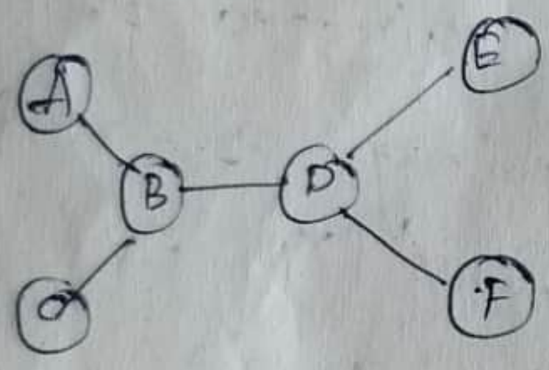
final solution.



. Structure of problems:

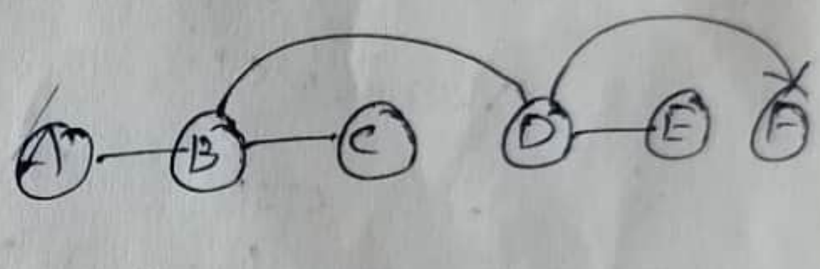
The structure of the problem as represented by the constraint graph, it can be used to find solutions very quickly.

- Most of applications are very general and are applicable to other problems beside csps.



(a)

The constraint graph of a tree structured csp



(b)

A linear ordering of the variables consistent with the tree with A as the root

This is known as topological sort of the variables

- It will have domain of {R, G, B} for all the variables

In the topological sort it is easy to find solution quickly. It is on order O(n)

- we can convert graph into a tree then we can implement topological sort technique

Directed Arc consistency & Topological sort

- A constraint graph is a tree when only two variables are connected by only one path.
- Any tree structure csp can be solved in linear time in the number of variables.
- A csp is defined to be directed arc consistent (DAc) under an ordering of variables $X_1 \dots X_n$ if and only if every X_i is arc consistent with each X_j for $j > i$
- An ordering of the variables such that Each variable appears after its parent in tree. such an ordering is called a topological sort.

Two ways to reduce constraint graph to trees.

1. Choose a cutset conditioning subset S of the csp's variables such that the constraint graph becomes a tree after removal of S. S is called a cycle cutset
2. for Each possible assignment to the variables in S that satisfies all constraints on S.

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

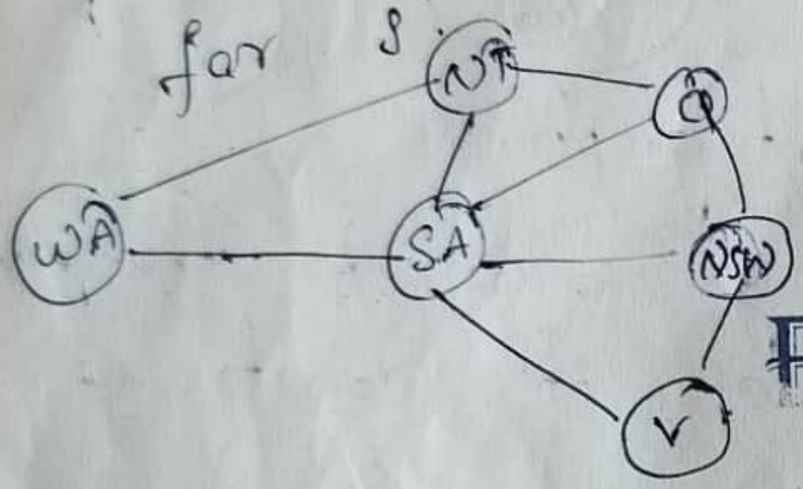
Name :
Year & Sem

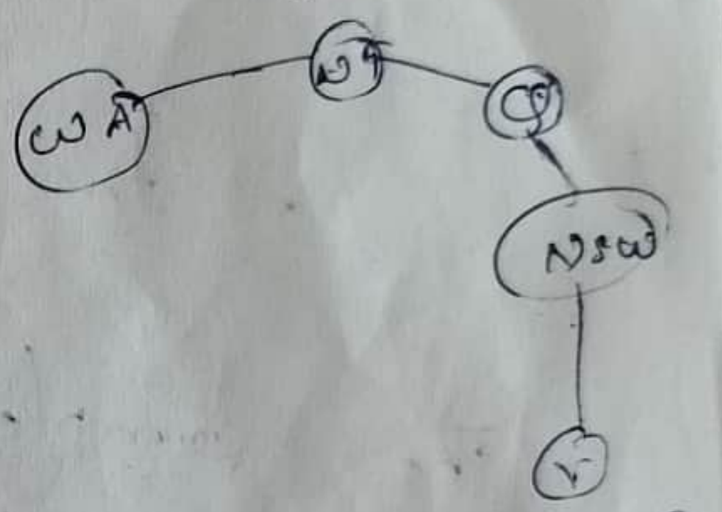
Subject : AI

H.T. No. :
Date :
Signature of Invigilator

a. Remove from the domain of the remaining variables any values that are inconsistent with the assignment for s and

b. If the remaining CSP has a solution, return it together with the assignment for s.





FB-SRIT

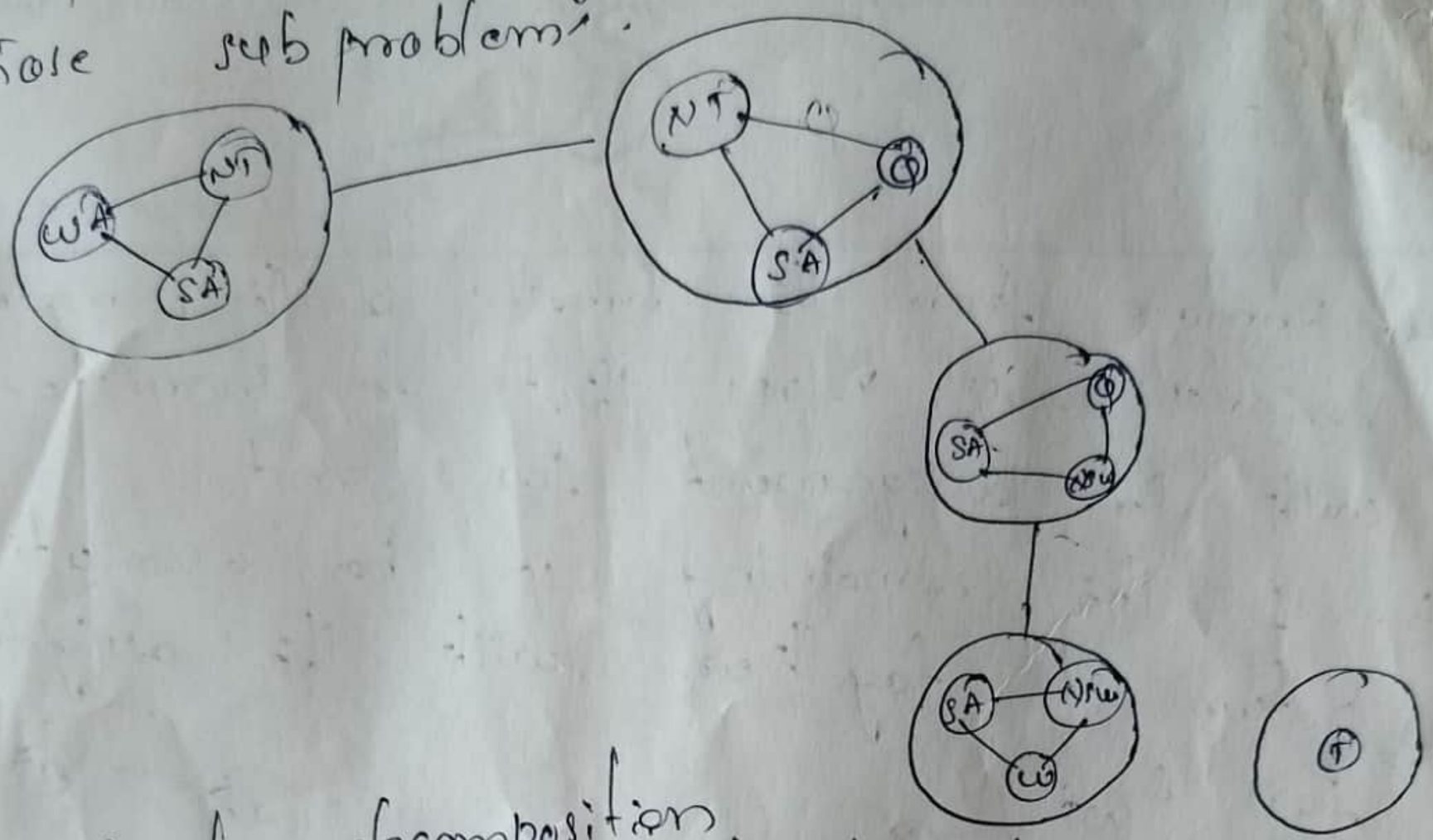
(4)

2. Tree decomposition:

It have to satisfy the following 3 requirements. (divide and conquer).

- Every variable in the original problem appears in atleast one of the subproblems.
- If 2 variables are connected by a constraint in the original problem, they must appear together (along with the constraint) in atleast one of the subproblems.
- If a variable appears in 2 subproblems in the tree, it must appear in every

subproblem along 16 path connecting those sub problems.



A tree decomposition of the constraint graph.

The complexity of solving a csp is strongly related to the structure of its constraint graph.

- Tree structured problems can be solved in linear time.
- cutset conditioning can reduce a general csp to a tree structured one and is very efficient if a small cutset can be found.
- Tree decomposition is efficient if the tree width of the constraint graph is small.
-

- Each state in a csp is defined by an assignment of values to some or all of the variables $\{x_i = v_i, x_j = v_j \dots\}$
- An assignment that does not violate any constraints is called a consistent or legal assignment.
- A complete assignment is one in which every variable is assigned, and a solution to a csp is a consistent, complete assignment.
- A partial assignment is one that assigns values to only some of its variables.

EB-SBIT

function BACKTRACK(assignment, csp) return a solution or failure

if assignment is complete return assignment

else SELECT unassigned variable, var // variable ordering

for each value in domain of a var // value ordering

if value is consistent with the assignment

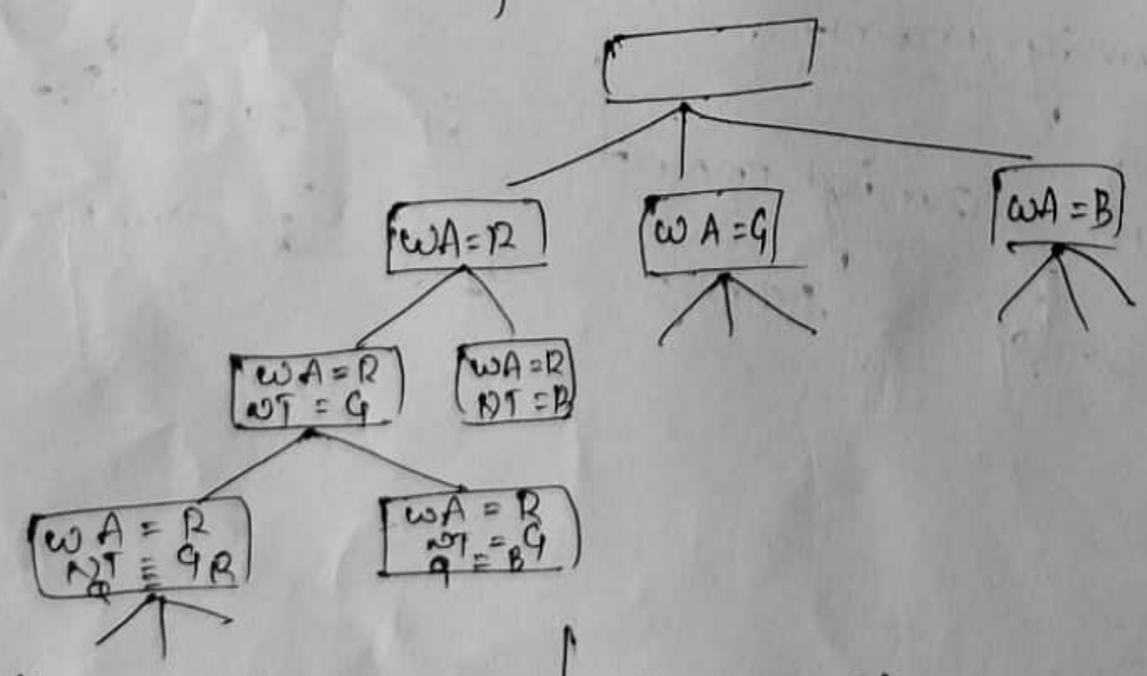
add {var = value} to assignment and

call BACKTRACK

if BACKTRACK succeeds, return assignment

else remove {var = value}

return failure



Initially assignment is null.

we need anchor condition b/c of that condition recursion will be terminated

if assignment completed then we get solution assignment as a solution.

how to select a variable can be done by variable ordering

Name :
Year & Sem

Subject : Al

H.T. No. :
Date :
Signature of Invigilator

assignment is the set we considered as perameter

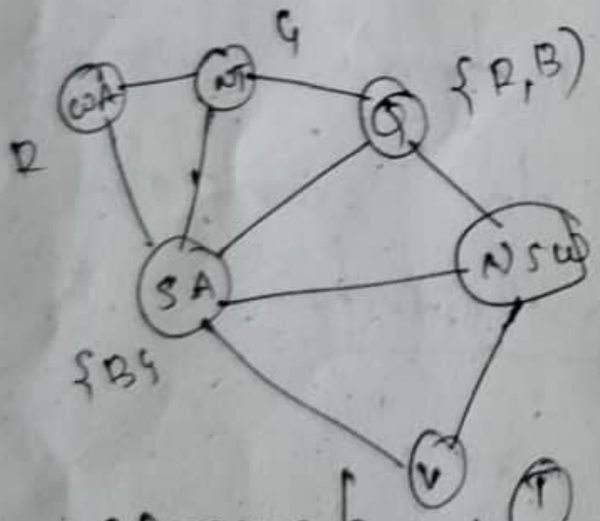
the value assigned to var moved over to the assignment.

variable ordering :

Minimum Remaining values (MRV) heuristic

. choose the variable with the fewest legal values

EB-SBIT



(Degree heuristic)
variable ordering

there SA is having the atleast value, so it is an most constrainted variable

The domain of all the variables in the map coloring is 3 {R, G, B} = 3 all are having equal domain, so it is not possible to select, in this case we find the degree of nodes i.e the highest degree of the node.

d(SA) = 5

It reduces the branching factor of future choices.

value ordering:

least constraining value heuristic:

prefers the value that rules out the fewest choices for the neighboring variables in the constraint graph.

- The heuristic is trying to leave the maximum flexibility for subsequent variable assignments.

from the above example Q is having {R, B} Q is left SA is having B constraint

Inference.

CSPs are a powerful tool for solving problems in AI and inference is a crucial technique for making these problems more tractable.

- By using inference, AI systems can efficiently find solutions to complex problems that involve constraints.

Common Inference Techniques are:

- forward checking
- Arc consistency
- path consistency

Benefits of Inference:

- Reduces search space
- Early failure detection
- Improved performance.

forward chec

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name : 115515 H.T. No. :

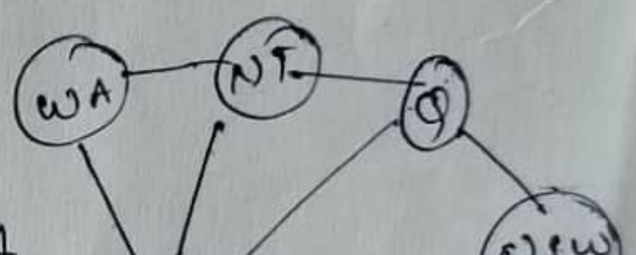
Year & Sem Subject : AI Date :

Signature of Invigilator

forward checking :

when a variable is assigned, forward checking ensures that all unassigned variables connected to the assigned variable by a constraint have their domains updated to remove values that are inconsistent with the assignment.

FR-SRIT



	WA	NT	Q	NSW	V	SA
initial domain	R G B	R G B	R G B	R G B	R G B	R G B
WA = R	R	G B	R G B	R G B	R G B	G B
Q = G	R	B	G	R B	R G B	B
V = B	R	B	G	R	B	

T

R G B
R G B
R G B
R G B

whenever a variable x is assigned, the f/w checking process establishes arc consistency for it.

- For each unassigned variable y that is connected to x by a constraint, delete from y's domain any value i.e inconsistent with value chosen for x.

1. solve the csp problem.

1. under the problem
2. set up equations and constraints
3. solve the puzzle.

```
  TWO
+ TWO
------
 FOUR
```

The unique letters in the puzzle
V = (T, W, O, F, U, R)
D = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9} set of possible values for each letter

(1) The above problem is a cryptoarithmetic puzzle written in a notebook, which is a type of constraint s.p

FB-SBIT

The goal is to assign digits to letters T, W, O, F, U, R such that the equation can be true i.e TWO + TWO = FOUR.

- Each letter/variable must assign a unique digit 0 to 9.
- leading zeros first letters (F, T) can not be 0.

(2) The equation can be represented column wise addition with carries (C_1, C_2, C_3) from right to left.

column$_1$ (units) : $O + O = R + 10 \cdot C_1$

column$_2$: $W + W + C_1 = U + 10 \cdot C_2$

column$_3$: $T + T + C_2 = O + 10 \cdot C_3$

column$_4$: $C_3 = F$

constraints:

```
  T W O
  T W O
+ ---
F O U R
```

```
  9 □ □
  9 □ □
+ ---
1 8 □ 0
```

0
1 - F
2
3
4
5
6
7
8
9 - T

$T \neq 0$ and $F \neq 0$ (leading digits)

carries $c_1, c_2, c_3 \in \{0, 1\}$

from $c_3 = F$

$F \neq 0$ so $F = 1$

$c_3 = 1$

$T + T + c_2 = O + 10 \cdot c_3$

$2T + c_2 = O + 10 \cdot 1$

$2T + c_2 = O + 10$

$T \neq 0$

$T = 7 \quad W = 3 \quad O = 4 \quad F = 1 \quad U = 6 \quad R = 8$

T	W	O	F	U	R
7	3	4	1	6	8

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem
Subject Allen
H.T. No. :
Date :
Signature of Invigilator

SEND
+ MORE
MONEY

S E M are leading variables

S ≠ 0 & M ≠ 0

M is the extra variables, carrying is

1. M = 1

V = {S, E, N D, M, O, R, Y}

D = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

The assig variables.

step1: starting from L.H.S the terms send & assign a digit which could give a satisfactory result.

FB-SBIT

M = 1

the next higher value from remaining 2, 3, 4, 5, 6, 7, 8 9

S = 9

```
  S        9
+ M      + 1
-----    ----
 M O      1 0
```

From this assignment S = 9 O is getting O

O = 0

step2: Next column, already O is 0. considering value next higher 8

```
  E        5
+ 0      + 0
-----    ----
  N        5
```

```
  7
+ 1
----
  6
```

E and N are getting same values so not possible.

carry 1.

$$\begin{array}{r} E \\ +\ 0 \\ \hline N \end{array}$$

$$\begin{array}{r} 5 \\ +\ 0 \\ \hline 6 \end{array}$$

$$\begin{array}{r} 8 \\ 0 \\ +\ p \\ \hline 8 \end{array}$$

step3: Next column. consider 6 for N and 8 for R

$$\begin{array}{r} N \\ +\ R \\ \hline E \end{array} \qquad \begin{array}{r} (1)\ 6 \\ +\ 8 \\ \hline (14) \end{array}$$ carry 1

step4: Next column

$$\begin{array}{r} D \\ +\ E \\ \hline \end{array} \qquad \begin{array}{r} 7 \\ +5 \\ \hline 12 \end{array}$$

S	E	N	D	M	O	R	Y
9	5	6	7	1	0	8	2

Ex:

$$\begin{array}{r} BASE \\ +\ BALL \\ \hline GAMES \end{array}$$

$$\begin{array}{r} 7483 \\ +\ 7455 \\ \hline 14938 \end{array}$$

$$\begin{array}{r} YOU\ R \\ +\ YOU \\ \hline HEART \end{array}$$

A = 4	
B = 7	
S = 8	
L = 5	
G = 3	
M = 9	

Y	9
O	4
U	2
R	6
H	1
E	0
A	3
~~R~~	
T	8

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem

Subject : AI

H.T. No. :
Date :
Signature of Invigilator

- Transpositions: It refers to a cache of previously seen game positions and their associated evaluations;
- It is used to avoid redundant searches in the game tree.
- This is essentially a form of memorization or dynamic programming applied to tree search.

FR-SRIT

Transposition Table These tables store the result of evaluations for specific game position,

- Minimum ~~value~~ ~~ordering~~ Remaining

Examples:

1.
  SEND
+ MORE

 MONEY

2.
  MONKEY
+ BANANA

3.
  water
+ jug
-----

while queue is not empty do.
if Revise (csp, X_i, X_j) then
if size of D_i = 0 then return false
for each X_k in X_i.neighbors - {X_j} do
add (X_k, X_i) to queue
return true.

function REVISE (csp, X_i, X_j) returns true iff we revise the domain of X_i

revised ← false
for each x in D_i do
if no value y in D_j allows (x, y) to the satisfy the constraint b/w X_i and X_j then
delete x from D_i
revised ← true
return revised.

1/4/25 (1)

Propositional Theorem proving: (2)

- we prove kb ⊨ α entailment can be done by theorem proving by applying rules of inference directly to the sentences in our knowledge. Base to construct a proof of the desired sentence without consulting models.

- logical equivalence:

Two sentences α and β are logically equivalent if they are true in the same set of models.

- we write this as $\alpha \equiv \beta$

we can easily show by truth tables that $p \wedge q = q \wedge p$ are logically equivalent.

example sample proof:

If John is not married he is a bachelor

$(\neg p \Rightarrow q)$

John is not a bachelor $(\neg q)$

prove John is married. i.e p

proof: S_1: $\neg p \Rightarrow q$ (implication elimination)

S_2: $\neg(\neg p) \vee q$ (Double negation elimination)

S_3: $p \vee q$ (By unit resolution)

S_4: p ..

conjunctive Normal form (CNF)

In propositional logic, the resolution method is applied only to those clauses which are disjunction of literals.

The following are the steps used to convert into CNF.

1. Eliminate Bi-conditional implication by replacing with conjuction

$$A \Leftrightarrow B = (A \rightarrow B) \wedge (B \rightarrow A)$$

2. Eliminate implication by replacing with conjunction

$$A \rightarrow B \equiv \neg A \vee B$$

3. In CNF, negation ($\neg$) appears only in literals, therefore we move it inward as.

$$\neg(\neg A) \equiv A \text{ (double negation elimination)}$$

$$\neg(A \wedge B) \equiv (\neg A \vee \neg B) \quad \text{Demorgan's law}$$

$$\neg(A \vee B) \equiv (\neg A \wedge \neg B) \quad \text{"}$$

4. finally using distribution law on the sentences and form the CNF as

$$(A_1 \vee B_1) \wedge (A_2 \vee B_2) \wedge (A_3 \vee B_3) \ldots \wedge (A_n \vee B_n).$$

Sum of products (SOP) or (POS)

when a sentence is in form of (SOP) or (POS) then it is called as CNF.

To apply resolution algorithm all the statements should be in CNF.

CNF
sentence
disjunctions
&
literals
sentence

A sentence expressed as a conjunction of disjunctions of literals is said to be in

CNF

A sentence in k-CNF has exactly k literals per clause.

- It can represented as

$(l_{1,1} \vee l_{1,2} \vee l_{1,3} \cdots l_{1,k}) \wedge (l_{2,1} \vee l_{2,2} \vee l_{2,3} \vee \cdots l_{2,k}) \wedge \cdots (l_{n,1} \cdots l_{n,k})$

- Every sentence can be transformed into 3CNF sentence that has equivelent set of models.

Question: Any propositional logic sentence is logically equivelent to the assertion that each possible world in which it would be false is not the case. From this observation prove that any sentence can be written in CNF.

Sol: prove the above assertions, we consider a simple conversion procedure.

-) we explain the procedure by converting considered the KB of wumpus world.

$S_1 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$; into CNF conditional Bi conditional eliminating for S_1.

steps as follows.

$S_2 : (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$

2. eliminating implication $\alpha \rightarrow \beta \equiv \neg\alpha \vee \beta$

$S_3 : (\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$

CNF requires $\neg$ to appear inside so we 'm inwards for S_3 by applying

Demorgan's law $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$

$\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$

$S_4 : (\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge P_{2,1}) \vee B_{1,1})$

apply $\wedge$ over $\vee$ distributivity law

$S_5 : (\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (P_{2,1} \vee B_{1,1})$

The considered sentence, now it is in CNF

Now we can apply resolution algorithm.

Resolution Algorithm:

- In resolution method, we use proof by Refutation / proof by contradiction technique to prove the given statement.
- To show $KB \models \alpha$, we show $(KB \wedge \neg\alpha)$ is unsatisfiable.

olution:

Algorithm:

- step$_1$: convert all sentence to CNF
  - write each clause down as a premise
- step$_2$: Negate the desired conclusion (converted to CNF)
  - add negated clause as a premise
- step$_3$: Apply resolution rule until either
  - Derive false (a contradiction)
  - can't apply any more.

$KB \models \alpha$ means KB implication α

$(KB \rightarrow \alpha)$

$\therefore \neg(KB \rightarrow \alpha)$

$\therefore \neg(\neg KB \vee \alpha)$

$\therefore \neg\neg KB \wedge \neg\alpha$

$\therefore KB \wedge \neg\alpha$

Examples

Question: 1. $p \vee q$
2. $p \rightarrow R$
3. $q \rightarrow R$
4. prove R.

step	formula	Derivation
1.	$p \vee q$	Given
2.	$\neg p \vee R$	(from 2. eliminating implies)
3.	$\neg q \vee R$	(from 3. ")
4.	$\neg R$	(negating given premise)

from premise ① and ②

$S_1 : P \vee Q$, $S_2 : \neg P \vee R$

By applying unit resolution law we get

$S_3 : Q \vee R$

consider with premise and love comperision with S_3.

$\neg R$ $Q \vee R$ with $\neg Q \vee R$

so we get $\neg Q$ by unit resolution $R \vee \neg R$.

$S_4 : \neg Q$

from S_3 and S_4 we get

R (By applying unit resolution)

R is proved

our assuming $(\neg R)$ is wrong.

Question : 1. $(P \to Q) \to Q$

2. $(P \to P) \to R$

3. $(R \to S) \to \neg(S \to Q)$

4. prove R.

sol : $\neg(P \to Q) \vee Q$

$\neg(\neg P \vee Q) \vee Q$

~~$\neg\neg P \wedge \neg Q \vee Q$~~ $(P \wedge \neg Q) \vee Q$

$(P \vee Q) \wedge (\neg Q \vee Q)$

$P \vee Q$

2. $(P \to P) \to$

$\neg(P \to P)$

$\neg(\neg P \vee$

$(P \wedge \neg$

$(P \vee$

3. (

$(p \to p) \to R$

$\neg(p \to p) \vee R$

$\neg(\neg p \vee p) \vee R$

$(p \wedge \neg p) \vee R$

$(p \vee R) \wedge (\neg p \vee R)$

3. $(R \to S) \to \neg(S \to Q)$

~~$\neg R \vee S$~~ $\neg(R \to S) \vee \neg(S \to Q)$

$\neg(\neg R \vee S) \vee \neg(\neg S \vee Q)$

$(R \wedge \neg S) \vee (S \wedge \neg Q)$

$(R \vee (S \wedge \neg Q)) \wedge (\neg S \vee (S \wedge \neg Q))$

$(R \vee S) \wedge (R \vee \neg Q) \wedge (\neg S \vee S) \wedge (\neg S \vee \neg Q)$

$(R \vee S) \wedge (R \vee \neg Q) \wedge (\neg S \vee \neg Q)$

1. $P \vee Q$
2. $P \vee R$
3. $\neg P \vee R$
4. $R \vee S$
5. $R \vee \neg Q$
6. $\neg S \vee \neg Q$
4. $\neg R$

from 4 and 1 $R \vee S$ and $\neg R$

8. S

from 6 and 8 $\neg S \vee \neg Q$ and S

9. $\neg Q$

from 1 & 9 prq, & 7q

s_{10}. p

from 3, & 10 ~~7pvR~~ & p

11. R

Hence we proved.

question: 3

1. The humidity is high or the sky is cloudy
2. If the sky is cloudy then it will rain,
3. If the humidity is high, then it is hot
4. It is not hot

Goal: If will rain

1. pvq
2. q → R
3. p → s
4. ~s
5. ~R

prove : R

solution proof:

6. 1. pvq
2. ~qvR
3. ~pvs
4. ~s
5. ~R

from 1, 2

6. pvR

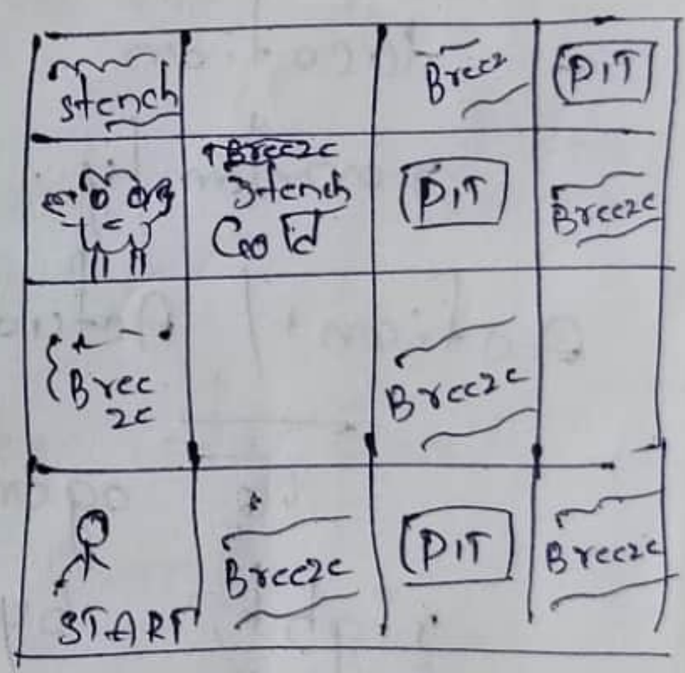
7. ~svR from 3, 6

8. R from 4, 7

9. . from 5, 8

proved ∎.

- wumpus world is a cave con 4*4 or 4/c rooms connected with passageways. 16 rooms which are connected with each other.
- so there are 16 rooms which are connected with each other.
- we have a KBA who will go f/w in this world.
- Re cave has a room with a beast which is called wumpus who eats anyone who enters the room.
- Re agent has FB-SBIT. 4 single arrow (wumpus) can kill with that arrow 2
- Some rooms contains pits that trap any agent that wonders in the room.
- Rere is a gold in a room which glitters.
- Re goal is to collect the gold and exit
- Re PEAS description of wumpus world
  ① performence ② environment ③ Action ④ sensors.
  ① when an agent comes out of the cave with gold he will be awarded +1000 reward points
  ② -1000 when he is eaten by wumpus or fallen into the pit



- -1 will be given when ever agent is flw direction, 90° or grab the gold will be recorded. (action)
- -10 for using an arrow, to shot the wumpus. to save agent.
- No game ends if the agent dies or agent come out of the cave.

Environment: 4*4 grid of rooms.
Initially it in room [1,1], facing towards right.
- location of wumpus & gold are choosen randomly. except first square [1,1]

Action / Actuator.
- The agent can move flw by left or right by 90°.
- The action Grab. the agent can pickup the gold when he was in that room.
- The action shoot. agent can shoot an arrow in a straight line when he finds an wumpus.
- The action climb can be used by agent when he want to come out of the room that can be done by [1,1].

Sensors. Five sensor.

① In the square containing the wumpus, and in the directly adjacent squares, the agent will perceive the stench

② When the agent is in the adjacent to pit the agent will perceive Breeze

3. when the agent wen in the square where the gold is present the Glitter will perceive by an agent.

4. when an agent walks into a wall
   b. if will perceive a Bump

5. when the wumpus was killed, if emits a woeful scream which can be perceived anywhere in the cave.

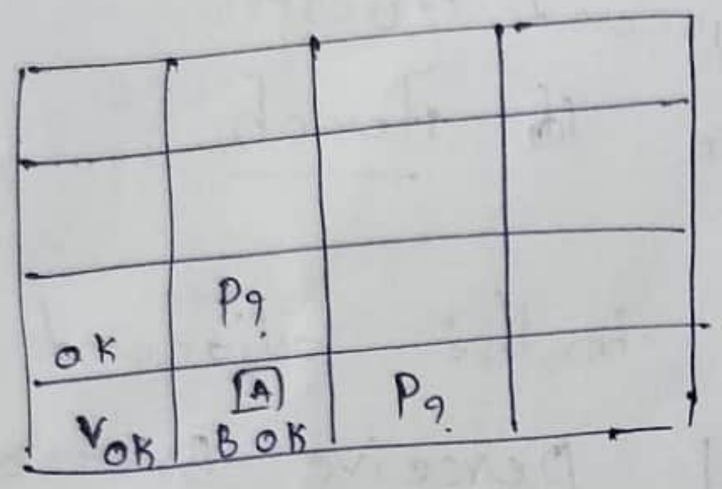
FB-SBIT

of agent into the pit

~~I will be~~ Example in Wumpus world 01

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1 [A]	2,1	3,1	4,1

A - Agent
B - Breeze
G - Glitter, Gold
ok - Safe square
P - pit
S - stench
V - visited
W - wumpus



Propositional logic

P.L is a formal system used to represent and reason about knowledge by breaking down it into simple declaretive sentence.

P.L < simple, declaretive

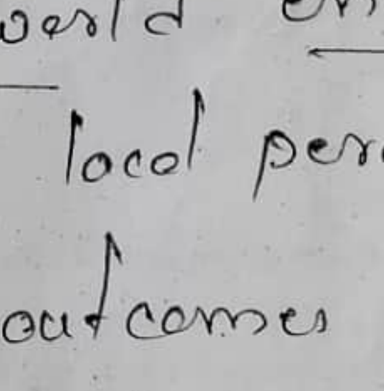
Environment:

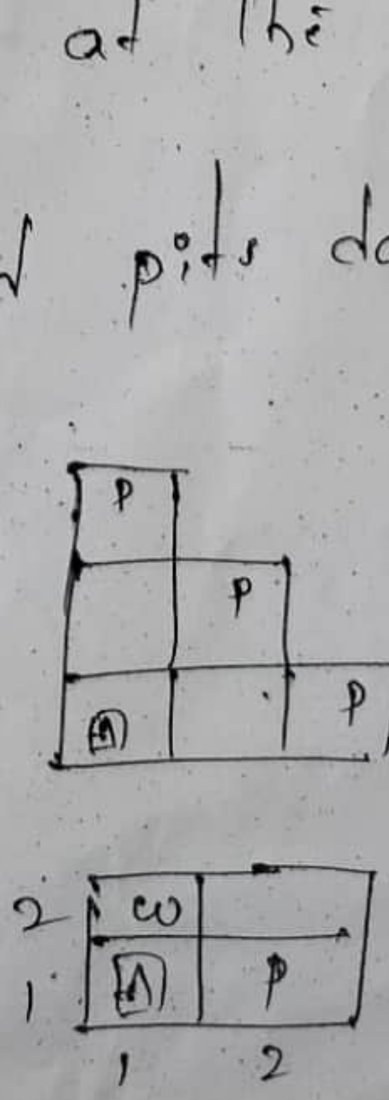
- square adjacent to wumpus are smelly
- square adjacent to ~~breeze~~ pit are breezy
- Glitter iff gold is in the same square
- shooting kills wumpus if you are facing it
- shooting uses up the only arrow
- Grabbing picks up the gold if in the same square
- Releasing drops the gold in the same square

EB-SBIT

characterization of wumpus world environment

- observable - partially, only local perception
- Deterministic - yes the outcomes are specified
- episodic - NO, sequential at the levels of actions
- static - yes, wumpus and pits do not move
- Discrete - yes
- single agent - yes
- other sticky situations





standard logical equivalences :

The symbols α, β and γ stand for arbitory sentences of propositional logic.

Two sentences are logically equivalent iff true in same models.

$\alpha \equiv \beta$ if and only if $\alpha \models \beta$ and $\beta \models \alpha$

EB-SBIT

$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$ commutativity of $\wedge$

$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$ commutativity of $\vee$

$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$ associativity of $\wedge$

$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$ " of $\vee$

$\neg(\neg\alpha) \equiv \alpha$ double negation elimination

$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$ contraposition

$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$ implication elimination

$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$ biconditional elimination

$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$ de morgan

$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$ de morgan

$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$ distributivity of $\wedge$ over $\vee$

$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$ distributivity of $\vee$ over $\wedge$

Equivalence is

- A sentence if valid, if it is true in all models.

Eg :- True, ① $A \vee \neg A$, ② $A \Rightarrow A$

$T \Rightarrow T = T$

$T \vee \neg T$

$T \vee F = T$

③ $A \wedge (A \Rightarrow B) \Rightarrow B$

$(T \wedge (T \Rightarrow F)) \Rightarrow F$

$(T \wedge F) \Rightarrow F$

$F \Rightarrow F = T$

In all the above the values are True.

. validity is connected to inference via the Deduction Theorem.

$KB \vdash \alpha$ iff $(KB \Rightarrow \alpha)$ is valid.

Inference: using the trained model to perform tasks.

Inference is the operational stage where a trained AI process new unknown or unseen data to make predictions, decisions or generate content.

that all individual

- $\alpha \equiv \beta$ if and only if $\alpha \models \beta$ and $\beta \models \alpha$
  (α is entailment β)
- A sentence is valid if it is true in all models
- valid sentences also known as tautologies
- Inference rules
- Some standard patterns of inference that can be applied to derive chains of conclusions that lead to desired goal.
- These patterns of inference are called inference rules.
- The best known rule is called
1. Modus ponens (MP)

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

If it is raining, Simley is inside
α - It is raining
β - Simley is inside
every time the result is β.
$\alpha \rightarrow \beta$ and α, the result is every time β.

$(\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$

(wumpus Ahead ∧ wumpus Alive) ⇒ shoot

(wumpus Ahead ∧ wumpus Alive) then shoo

can be inferred.

- Modes Tollens (MT) it is an another inference rule.

example:

- If it is raining. then Rahul is inside
- Rahul is not inside

α - It is raining

β - Rahul is inside

¬β - Rahul is not inside

α → β and ¬β

∴ ¬α

- And elimination (AE)

These rules can then be used in any particular instances where they apply, generating sound inferences without the need for enumerating models.

And elimination

- from a conjunction, any of the conjuct can be inferred

$$\frac{\alpha \wedge \beta}{\alpha}$$

- example

- (wumpusAhead ∧ wumpusAlive), wumpusAlive

can be inferred:

- By considering the truth values of α & β one can show easily that Modus ponens and And-elimination are sound once for all

- Bicondidional elimination

The equivalence for bicondidional elimination yields the 2 inference rules

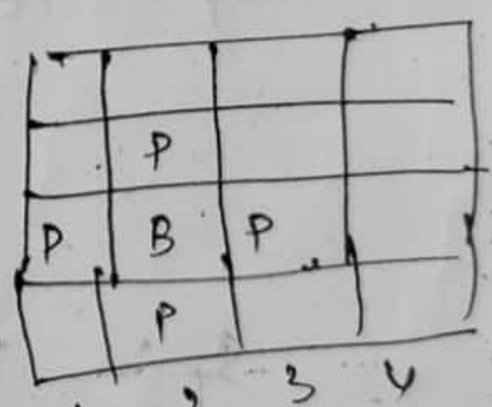
$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \qquad \frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

FB-SRIT

- example proof by deduction (wumpus world)

- knowledge Base

$S_1 : B_{22} \Leftrightarrow (P_{21} \vee P_{23} \vee P_{12} \vee P_{32})$



$S_2 = \neg B_{22}$ By rule observation

- Inferences:

S_3:

→ In S_1 removing the Bicondidional we are implementing ⇒ and ∧

$\alpha \Leftrightarrow \beta = (\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$

S_3 : $(B_{22} \rightarrow (P_{21} \vee P_{23} \vee P_{12} \vee P_{32})) \wedge ((P_{21} \vee P_{23} \vee P_{12} \vee P_{32}) \rightarrow B_{22})$

(This is Bi-conditional elimination removing and (∧) elimination)

- we can take any one consider it as S_4 : $\frac{(\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)}{p}$ and elimination $\frac{}{q}$ we can take any one pair eliminate

S_4 : $((P_{21} \vee P_{23} \vee P_{12} \vee P_{32}) \Rightarrow B_{22})$ contra rep.

S_5 : for position i.e we are including negation in the statement

S_5 S_6 : $(\neg B_{22} \Rightarrow \neg(P_{21} \vee P_{23} \vee P_{12} \vee P_{32}))$

S_6 : considering $\neg(P_{21} \vee P_{23} \vee P_{12} \vee P_{32})$ and S_2 and S_6 applying modus ponens inference rule.

$\alpha \Rightarrow \beta \wedge \alpha = \beta$

S_7 : $(\neg B_{22} \Rightarrow) \neg(P_{21} \vee P_{23} \vee P_{12} \vee P_{32})) \wedge \neg B_{22}$

S_7 : $= \neg(P_{21} \vee P_{23} \vee P_{12} P_{32})$

S_7 (By Demargen's law

S_7 : $\neg P_{21} \wedge \neg P_{23} \neg P_{12} \wedge \neg P_{32}$

$\neg B_{22}$ By contradiction we proved.

ADDITIONAL ANSWER SHEET

Signature of Invigilator Page No. :

Sample rules of inferences:

- modus ponens : $(\alpha \Rightarrow \beta, \alpha) \vdash \beta$
- And elimination $(\alpha \wedge \beta) \vdash \alpha$, $(\alpha \wedge \beta) \vdash \beta$
- And introduction $\{\alpha, \beta\} \vdash \alpha \wedge \beta$
- or introduction $\{\alpha\} \vdash \alpha \vee \beta$
- Double negation elimination $\{\neg\neg\alpha\} \vdash \alpha$
- Implication elimination $\{\alpha \Rightarrow \beta\} \vdash \neg\alpha \vee \beta$
- unit resolution $\{\alpha \vee \beta, \neg\beta\} \vdash \alpha$

FB-SRIT

- Resolution : $\{\alpha \vee \beta, \neg\beta \vee \gamma\} \vdash \alpha \vee \gamma$

A tautological implication means all conclusion where result should be true.

sample proof

If John is not married he is a bachelor

$(\neg p \Rightarrow q)$

p - John married

q : he is bachelor.

$\neg q$ John is not bachelor

$\neg p \Rightarrow q$ (implication elimination

~~$\neg p \vee q$~~

$\alpha \Rightarrow \beta = \neg\alpha \vee \beta$ (Implication elimination)

$\neg\neg p \vee q \equiv p \vee q$ (double negation removing)

$= p \vee q \quad \neg q$

$= p$ unit resolution.

$((\neg p \Rightarrow q) \wedge \neg q) \Rightarrow p$

→ we can also check validity.

- Simple Knowledge Base

propositional symbol representation

- let P_{ij} be true if there is a pit in location $[i, j]$

- let B_{ij} be true if there is a pit in location $[i, j]$

The following are some of the KB.

$R_1 : \neg P_{1,1}$ no pit - T

$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

$R_4 : \neg B_{1,1}$

$R_5 : B_{2,1}$

The KB consists of sentences R_1 through R_5

It can also be considered as a single sentence b/c in all cases it is true

$R_1 \wedge R_2 \wedge R_3 \wedge R_4 \wedge R_5 = T$ b/c it awards

$T \wedge T \wedge T \wedge T \wedge T = T$

that of all individual statements are true.

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem : 2-10 Subject : AI Lab

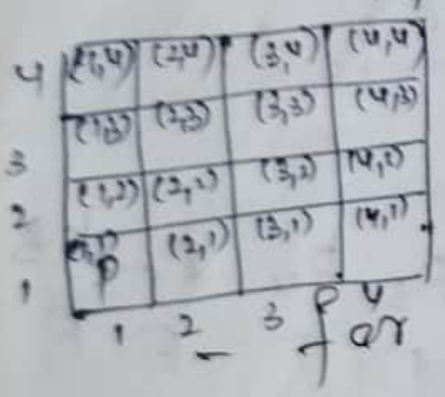
H.T. No. : 1430
Date :
Signature of Invigilator

- A simple knowledge Base

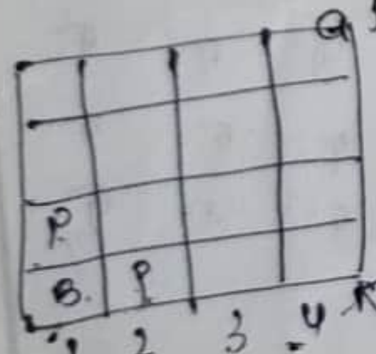
We have defined the semantics for p. logic, we can construct a KB for the wumpus world

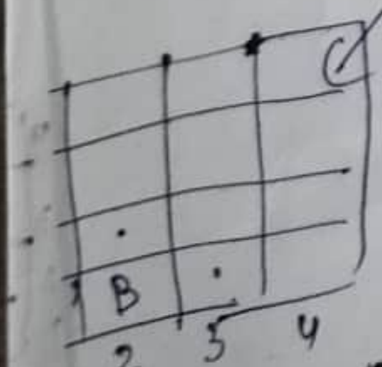
- let $P_{i,j}$ be true if there is a pit in $[i,j]$
- let $B_{i,j}$ be true if there is a Breeze in $[i,j]$
- The KB includes the following sentences each one labeled for our convenience.

FR-SRIT



for





If there is no pit in [1,1]:

$B_1 : \neg P_{1,1}$ ~~$\neg P_{1,1}$~~

P - pit
B - Breeze

example if there is a Breeze in following rooms we can represent the following wumpus world.

$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

The following sentences are true in all wumpus world.

$R_4 : \neg B_{1,1}$ (B is not present so $\neg B$ (¬F) means True)

$R_5 : B_{2,1}$ (B is present so T).

R_1, R_2, R_3, R_4, R_5 are sentences / propositions symbols

- The knowledge consists of sentences R1 through R5
- It can also be considered as a single sentence.
- The conjunction $R_1 \wedge R_2 \wedge R_3 \wedge R_4 \wedge R_5$ b/c it asserts that all the individual sentences are true.

Inference:

- By considering all the values $B_{1,1}$, $B_{2,1}$, $P_{1,1}$ $P_{1,2}$ $P_{2,1}$ $P_{2,2}$ $P_{3,1}$ their rooms can be representing with following sentences.
- with the above 7 propositions we can get on $2^7 = 128$ possible models.
- In these 3 models KB is true

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
F	F	F	F	F	F	F	T	T	T	T	F	F
⋮												
F	T	F	F	F	F	F	T	T	T	T	T	T
F	T	F	F	F	F	T	T	T	T	F	T	F
⋮												
F	T	F	F	T	F	F	T	T	F	T	T	T
F	T	F	F	T	F	T	T	T	F	F	T	F
⋮												
F	T	F	T	F	F	F	T	T	T	T	T	T
F	T	F	T	F	F	T	T	T	T	F	T	F
⋮												
F	T	F	T	T	F	F	T	T	F	T	T	T
F	T	F	T	T	F	T	T	T	F	F	T	F
⋮												
F	T	T	F	F	F	F	T	T	T	T	T	T
F	T	T	F	F	F	T	T	T	T	F	T	F
⋮												
F	T	T	F	T	F	F	T	T	F	T	T	T
F	T	T	F	T	F	T	T	T	F	F	T	F
⋮												
F	T	T	T	F	F	F	T	T	T	T	T	T
F	T	T	T	F	F	T	T	T	T	F	T	F
⋮												
F	T	T	T	T	F	F	T	T	F	T	T	T
F	T	T	T	T	F	T	T	T	F	F	T	F
⋮												
T	T	F	F	F	F	F	T	T	T	T	T	T
T	T	F	F	F	F	T	T	T	T	F	T	F
⋮												
T	T	F	F	T	F	F	T	T	F	T	T	T
T	T	F	F	T	F	T	T	T	F	F	T	F
⋮												
T	T	F	T	F	F	F	T	T	T	T	T	T
T	T	F	T	F	F	T	T	T	T	F	T	F
⋮												
T	T	F	T	T	F	F	T	T	F	T	T	T
T	T	F	T	T	F	T	T	T	F	F	T	F
⋮												
T	T	T	F	F	F	F	T	T	T	T	T	T
T	T	T	F	F	F	T	T	T	T	F	T	F
⋮												
T	T	T	F	T	F	F	T	T	F	T	T	T
T	T	T	F	T	F	T	T	T	F	F	T	F
⋮												
T	T	T	T	F	F	F	T	T	T	T	T	T
T	T	T	T	F	F	T	T	T	T	F	T	F
⋮												
T	T	T	T	T	F	F	T	T	F	T	T	T
T	T	T	T	T	F	T	T	T	F	F	T	F

Algorithm for entailment in propositian logic

function TT-ENTAILS ? (KB, α) returns true or fole
inputs: KB, the knowledge bose, a sentence
in propositianal logic
α, the query, a sentence in p.l
symbols ← a list af propositian
symbols in KB and α
return TT-CHECK-ALL (KB, α, symbols, [])

FR-SRIT

function TT-CHECK-ALL (KB, α, symbols, model)
returns true or fole
if EMPTY ? (symbols) then
if PL-TRUE ? (KB, model) then return
PL-TRUE ? (α, model)
else return true
else do
p ← FIRST (symbols); rest ← REST (symbols)
return TT-CHECK-ALL (KB, α, rest,
EXTEND (p, true, model) and
TT-CHECK-ALL (KB, α, rest, EXTEND
(p, false, model)

- This algorithm specifies, ~~prope~~ deciding propositional entailment
- TT stands for truth-table
- pl-true? returns true if a sentence holds within a model.
- The variable model represents a partial model -

Sound:
The algorithm is sound, b/c it implements directly the definition of entailment.

complete:
The algorithm is complete b/c it works for any KB and α and always terminates
- There are only finitely many models to examine.
- finitely many means not few, if KB and α contain n symbols in all, then there are 2^n models.
- The time complexity of algorithm is $O(2^n)$
- space complexity is $O(n)$.

Sec-B (not submitted after 2 warnings) 2.11

LE- 505, 506, 502

5f8

5H2

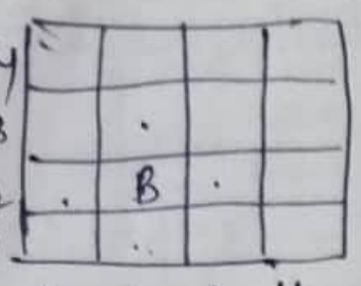
5I4

5f4

5f9'

5I2

5G6



S_1 : $B_{22} \Leftrightarrow P_{21} \vee P_{23} \vee P_{12} \vee P_{32}$ (B is

S_2 : $\neg B_{22}$

S_3 by applying bicondition. (logical equivel

$\alpha \Leftrightarrow \beta = (\alpha \rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$

S_3 $\{B_{22} \rightarrow (P_{21} \vee P_{23} \vee P_{12} \vee P_{32})\} \wedge \{(P_{21} \vee P_{23} \vee P_{12} \vee P_{32}) \rightarrow B_{22}\}$

And elimination

$\alpha \wedge \beta = \alpha$ or $\alpha \wedge \beta = \beta$

S_4 $\{(P_{21} \vee P_{23} \vee P_{12} \vee P_{32}) \Rightarrow B_{22}\}$ (By eq logical equivalence)

apply contra position S_4 $\alpha \Rightarrow \beta = \neg\beta \rightarrow \neg\alpha$

S_5 : $\neg B_{22} \Rightarrow \neg(P_{21} \vee P_{23} \vee P_{12} \vee P_{32})$

$\neg B_{22} = \alpha$, β

apply modus ponens to S_2 and S_5 ~~$(\alpha \Rightarrow \beta) \alpha \beta$~~ $(\alpha \Rightarrow \beta, \alpha)$ give β

S_6 : $\neg(P_{21} \vee P_{23} \vee P_{12} \vee P_{32})$

S_7 : $\neg P_{21} \wedge \neg P_{23} \wedge P_{12} \wedge P_{32}$

proved.

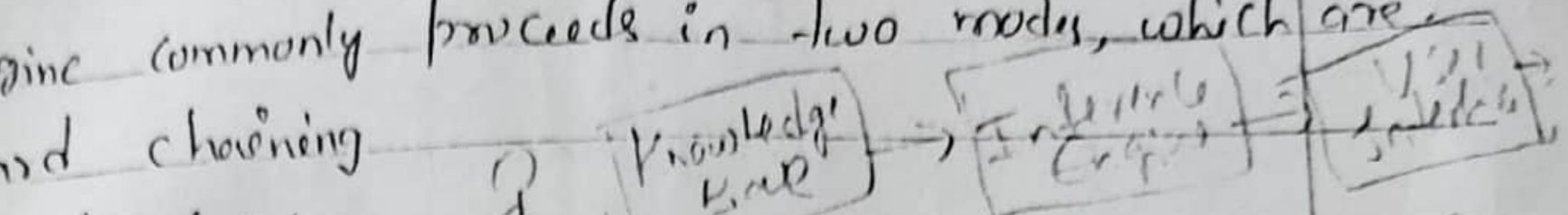
Inference Engine:

The Inference Engine is the component of the intelligent in A.I, which applies logical rules to the Knowledge Base to infer new information from known facts.

- The first Inference engine was part of the expert sys.

Inference engine commonly proceeds in two modes, which are:

- Forward chaining
- Backward chaining.



Two important clauses of Inference Engine is

1.- Horn clause 2.- Definite clause.

Horn clauses and definite clause:- are the forms of sentences, which enables knowledge base to use a more restricted and efficient inference algorithm.

Logical Inference algorithm:- use forward & backward chaining approaches, which require KB in the form of the First-Order-Logic clauses.

Definite clause:- A clause which is disjunction of literals with exactly one positive literal is known as a definite clause or strict horn clause. $\neg A \vee \neg B \vee C$

Horn clause:- A clause which is a disjunction of literals with at most one positive literal is known as horn clause. Hence all the definite clauses are horn clauses.

Ex: $(\neg p \vee \neg q \vee K)$. It has only one positive literal K.

- It is equivalent to $p \wedge q \rightarrow K$. Unnegated literal

$p \wedge q \rightarrow K$ - definite clause

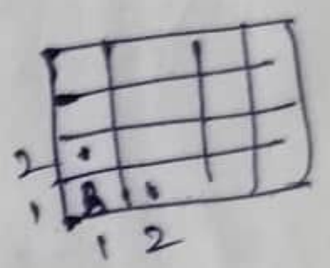
$\neg p \vee \neg q \vee K$ - Horn only not definite.

$(p \wedge q) \rightarrow K$.

- prove there is no pit in location 1, 2

$\neg P_{1,2} \wedge \neg P_{2,1}$ or $\neg P_{1,2}$ or

→ consider R_2 and apply Bi-conditional.



R_6 : $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

R_6 : $\{B_{1,1} \rightarrow (P_{1,2} \vee P_{2,1})\} \wedge \{(P_{1,2} \vee P_{2,1}) \rightarrow B_{1,1}\}$

apply and elimination to R_6

EB-SBIT

R_7 : $\{(P_{12} \vee P_{21}) \rightarrow B_{11}\}$

apply logical equivalence contradiction to R_3.

$\alpha \rightarrow \beta \equiv \neg\beta \rightarrow \neg\alpha$

R_8 : $\neg B_{11} \rightarrow \neg(P_{1,2} \vee P_{2,1})$

R_4 : $\neg B_{11}$

Modus ponen

Apply inference rule to R_4 and R_8

R_8 : $\neg(P_{1,2} \vee P_{2,1})$

Apply de-morgan to R_8

R_9 : $\neg P_{1,2} \wedge \neg P_{1,2}$

no pit in the room.

proved

∎.

Conjunctive normal form

R_6 : $\neg B_{1,2}$

R_7 : $B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$

R_8 : $\neg P_{2,2}$

R_9 : $\neg P_{1,3}$ Biconditional elimination R_3 with R_5

R_{10} : $P_{1,1} \vee P_{2,2} \vee P_{3,1}$ R_{10} (resolved with R_8).

R_{11} : $P_{1,1} \vee P_{3,1}$

R_{12} : $P_{3,1}$ (resolve R_1 with R_{11}

(It is the procen of using a model to make predictions or solve tasks

Inference engine:

It is the component of the intelligent system in A.I, which applies logical rules to the KBase to infer new information from known facts.

- The first inference engine was part of the expert system.

- Inference engine commonly proceeds in 2 modes.

. forward chaining

. B/w chaining

There are 2 types of clauses in inferena Horn clause and definite clause

These are the forms of sentences, which enables KBase to use a more restricted and efficient inference algorithm.

- logical inference algorithms use F/w and b/w chaining approaches, which uses KB in the form of the first order definite clause

EB-SBIT

Definite clause:

A clause which is a disjunction of literals with exactly one positive literal is known as definite clause or strict horn clause.

Horn clause:

A clause which is a disjunction of literals with atmost one positive literal is known as horn clause

- Hence all definite clauses are horn clauses

ex:- $(\neg p \vee \neg q \vee k)$ - It has only one positive literal k.

- It is equivalent to $p \wedge q \rightarrow k$

clause form: Statements are converted into a standard form called clause form, which are disjunctions of literals.

Resolution: AI resolution refers to a proof procedure particularly in propositional and first-order logic, used to determine the validity of a logical formula by proving it unsatisfiable through a process of contradiction.

logical Agent:

Agent can represent the knowledge of its world, goals and its current situation.

logical Agent:

It is having a collection of sentences

Each sentence contains a logic. so called as logic sentences.

- By using logical sentences the agent decides what to do by inferring knowledge if gives conclusions

- The conclusions are achieved by certain action or set of actions is appropriate to achieve its goals.

- knowledge and reasoning are important to logical agents b/c they enable succesful behaviours to achieve a desired goal.

EB-SBIT

knowledge Based Agent:

The central component for KBA is knowledge Base.

- KB contains a set of sentences in a formal language

. Sentences are exprened using a knowledge representation language.

Two generic functions:

① Tell - add new sentences (facts) to the knowledge Base.

" Tell it what it needs to know"

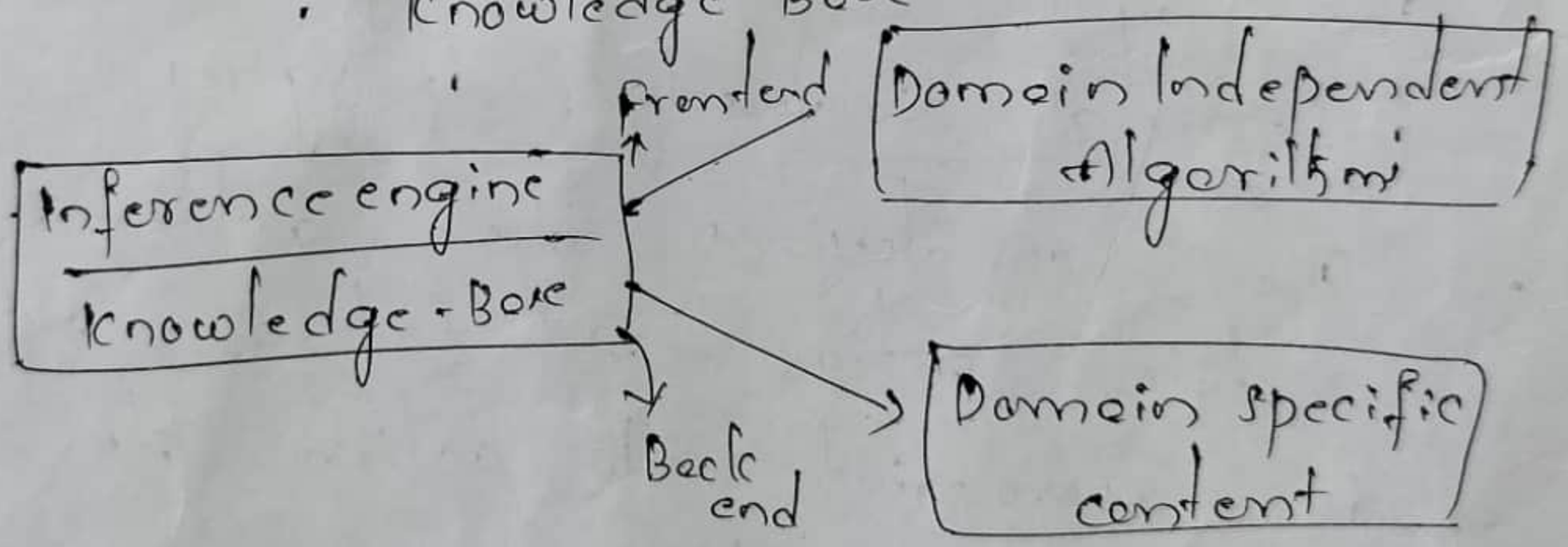
② Ask - query what is known from the KB

" Ask what to do next"

KBA : It is divided into 2 part

. Inference engine

. Knowledge Base



- Knowledge Bole it collection of sentences i.e Domain specific content.

- Inference engine if it on domain independent algorithm.

- KB agent represents
  - state and action.

- Incorporate new percepts.

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem

Subject : AI

H.T. No. :
Date :
Signature of Invigilator

- update internal representation of the world
- (Derive) Deduce hidden properties of the world
- (Derive) Deduce appropriate actions.

KB Algorithm

function KB-AGENT (percept) returns an action t

static : KB, a knowledge Base

t, a counter, initially 0, indicating time

TELL (KB, MAKE-PERCEPT-SENTENCE (percept, t))

action ← Ask (KB, MAKE-ACTION-QUERY (t))

TELL (KB, MAKE-ACTION-SENTENCE (action, t))

t ← t+1

return action

FB-SBIT

- KBA are 2 types ← Declarative, procedural

Declarative - we can build a KBA simply by TELLING it what it needs to know.

procedural : Encode desired behaviour directly as program code

- minimizing the role of explicit representation and reasoning can result in a much more efficient system.
- The agent maintains a knowledge Base, KB, which may initially contain some background knowledge.

- Designing the representation language to make it easy to express this knowledge in the form of sentences, which simplifies the construction problem enormously. This is called declarative approach.

propositional logic works with statements called propositions. Each can be true / false.
- proposition represents fact or conditions about a situation.
- we use symbols to represent the propositions and logical operators to connect the propositions.

	CSE-B	F	F	P => Q	P <=> Q
submitted today - 19/3/25		F	T	T	F
5G6	LE 502, 507, 501	T	F	F	F
5F8		T	T	T	T
5I4					
5G3					
5F9					

SWARNA BHARATHI
Name:
Year & Sem

explicit
com
system.
result
Bool

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem

Subject : AI

H.T. No. :
Date :
Signature of Invigilator

(p.c) propositional logic: It is a very simple logic, statements are made by propositions.

- pro.c can also be called as B.C
- the sen/sts can be declarative .st i.e if can be true or false but not both.
- questions, opinions, commen r not allowed in p.c.

EB-SBIT

students are studying college True pro
5+3=8 (" ')
5+4=10 (false ")
where are you going? (f.p)
some students r intelligent (fake.p)
b/c it is have true and false both
print("Hello")

syntax - defines allowable sentences.
1. atomic sen 2. complex.

1. Ae individual syntactic elements consist of a single proposition symbol.
each symbol stands for proposition that can be true or false
we will use upper case symbols P, Q, R, S
Ex: Wumpus W[1,3]

complex sentences - constructed from simpler sentences using connective
not (¬) called negation.
- A literal is either atomic (positive literal) or negated (negative literal)
- ∧ (and) - conjunction its parts are the conjuncts
- ∨ (or) - a disjunction of the disjuncts
- ⇒ (implies) is called implication (or conditional)
- ⇔ (if and only if) the sentence is bi

A formal Grammar of P. Logic

- A BNF (Backus - Naur Form) grammer of sentences in P.L

	① Atomic. S	② C.S
Sentence →	AtomicSentence	ComplexSentence
AtomicSentence →	True | False | symbol	
symbol →	P | Q | R ...	
complex.s →		¬ sen
		| (sent ∧ s)
		| (sent ∨ s)
		| (se ⇒ sen)
		| (sen ⇔ sen)

semantics defines the rules for determining the truth of a sentence w.r.t particular model.

Truth tables for 5 logical connectives

P	Q	¬P	P∧Q	P∨Q	P ⇒ Q (P → Q) if then	P ⇔ Q bi-conditional (if and only if)
F	F	T	F	F	T	T
F	T	T	F	T	T	F
T	F	F	F	T	F	F
T	T	F	T	T	T	T

- Entailment The relation b/w a sentence and another sentence that follows from it, and implementing a simple algorithm for logical inference, is called entailment

ex: The sentence can be $\neg P \vee Q \wedge R \Rightarrow S$

$((\neg P) \vee (Q \wedge R)) \Rightarrow S$

- logical equivalence :

Two sentences α and β are logically equivalent if they are true in the same set of models.

- we write this as $\alpha \equiv \beta$

- $\alpha \equiv \beta$ if and only if $\alpha \Rightarrow \beta$ and $\beta \Rightarrow \alpha$

$\alpha \models \beta$ and $\beta \models \alpha$

validity : checking the validity of $(KB \Rightarrow \alpha)$

- for any sentences α and β, $(\alpha \Rightarrow \beta)$ if and only if, the sentence $(\alpha \Rightarrow \beta)$ is valid

- satisfiability : A sentence is satisfiable, if it is ~~sentence~~ $(\alpha \Rightarrow \beta)$ true in some model.

- A sentence α is true in a model 'm' ~~model~~ then, we say that m satisfies α, or m is a model of α

- In p.logic a model simply fixes the truth value of every propositian symbol.
- m_1 = ($P_{1,2}$ = false, $P_{2,2}$ = false, $P_{3,1}$ = true).
- The semantics for p.logic must specify how to compute the truth value of any sentence, given a model.
- This is specifically done recursively.
- All sentences are constructed from atamic sentences and the five connectives.
- To specify how to compute the truth of atamic sentences and truth of sentences formed with each of the five connectives.
- Atomic sentences are easy.
- Complex sentences

~~for~~ If any sentence s and any model m the sentence ¬s is true in 'm' if and only if s is false in m

Truth table for logical sentences:

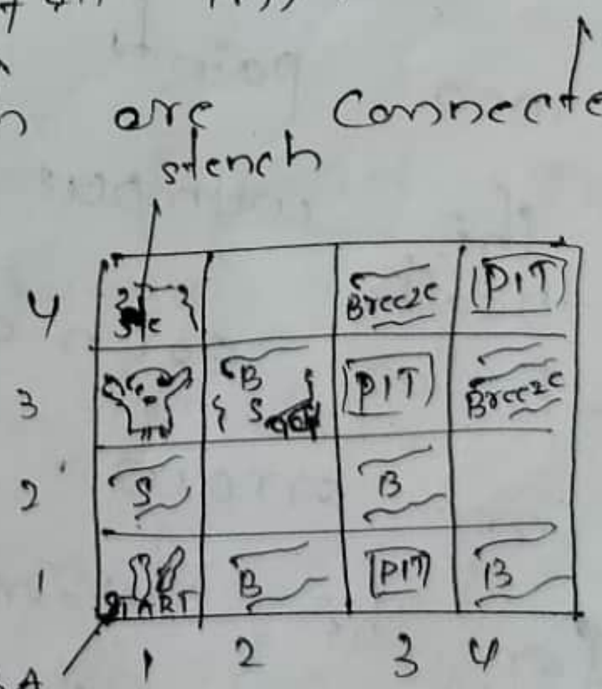
- A sentence such as $(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$ is called implication or conditional.

If (Condition (p) then q, 'p' can be called an antecedent / premise q is called an Consequent

(stranger) (monster)

Wumpus world is a cave consisting of rooms connected by passage ways. size of 4/4

- A beast that eats any one who enters its room.
- wumpus that hides some where in the cave.
- The wumpus was shot by an agent
- The agent has only one arrow.
- some rooms contain bottomless pits that will trap anyone who wonders into these rooms (except for the wumpus, which is too big to fall in).
- Total 16 rooms which are connected with each other.



A typical wumpus world. The agent is in the bottom left corner.

- we have a knowledge BA who will go forward in this world.

EB-SBIT

- The cave has a room with beast which is called wumpus, who eats anyone enters the room.
- pit - A large hole in the ground
- stench - a strong & very unpleasant smell
- occasionally there is a heap of gold in a room.
- The goal is to collect the gold and exit the world without being eaten
- PEAS description of wumpus world.

performance measure:

- +1000 reward points if the agent comes out of the cave with the gold.
- -1000 points penalty for being eaten by the wumpus or falling into the pit.
- -1 for each action, and -10 for using an arrow.
- For the movement that is moving up, forward 90° -1 will be awarded
- when an agent uses an arrow -10 will be reduced.

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem

Subject :

H.T. No. :
Date :

Signature of Invigilator

- The game ends if either agent dies or came out of the cave.

- Environment :

A 4*4 grid of rooms.

- Initially the agent was present in the room [1,1] square facing towards the right

EB-SBIT

The remaining they can present anywhere in the room.

- Actions / Actuators :

- The agent can move forward, Turn left by 90° or Turn right by 90°, to make a move from one room to other.
- The action Grab can be used to pick up the gold if it is in the same square as the agent.
- The action shoot can be used to fire an arrow in a straight line in the direction the agent is facing.

The action climb can be used to climb out of the cave, but only from square [1,1]. The can perform this by having in the place [1,1].

Sensors:

The agent is equipped with 5 sensors.

- In the room containing the wumpus and in the directly (not diagonally) adjacent squares, the agent will percept a stench, it means the wumpus was present.

Acting to the above figure when the agent was in the location [1,2], the agent can stench that the wumpus was in the adjacent room i.e [1,3]

- when the agent was in [2,2] he can't stench b/c it is diagonal.

2nd sensor is Breeze,

- The agent will perceive the Breeze when he was in [2,1] the adjacent is PIT [3,1]

- when the agent was in [2,2] he can't Breeze the PIT, it is diagonal in the place [3,3]

SWARNA BHARATHI
Name:
Year & Sem

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem

Subject : AI

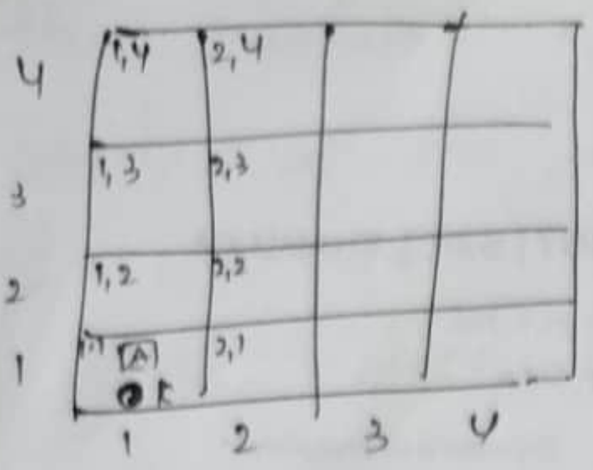
H.T. No. :
Date :
Signature of Invigilator

- The 3rd sensor is glitter, the agent will perceive the glitter, when the gold is present in the room.
- The agent can perceive when he was in the room [2,2] [2,3]
- when agent walks into wall, it will perceive a Bump.
- when wumpus is killed, it emits a woeful scream that can be perceived anywhere in the cave.
- The output of five sensors will be given as input to the agent program, based on the present five symbols.

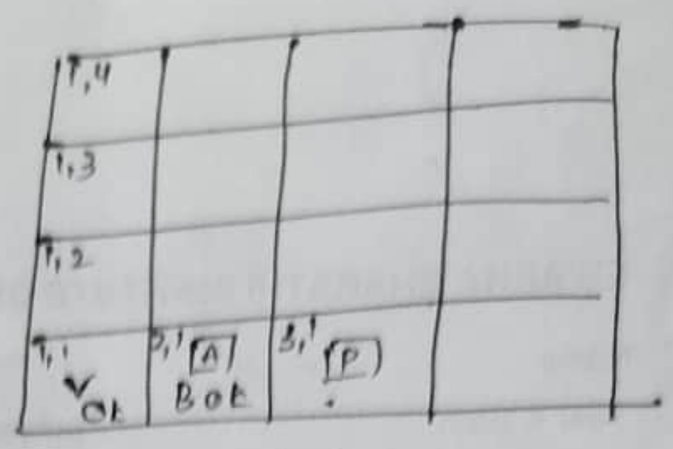
Variables used

A - Agent
B - Breeze
G = Glitter, Gold
ok - safe square
P - pit
S - stench
V - visited
W - wumpus

EB-SBIT



(a)

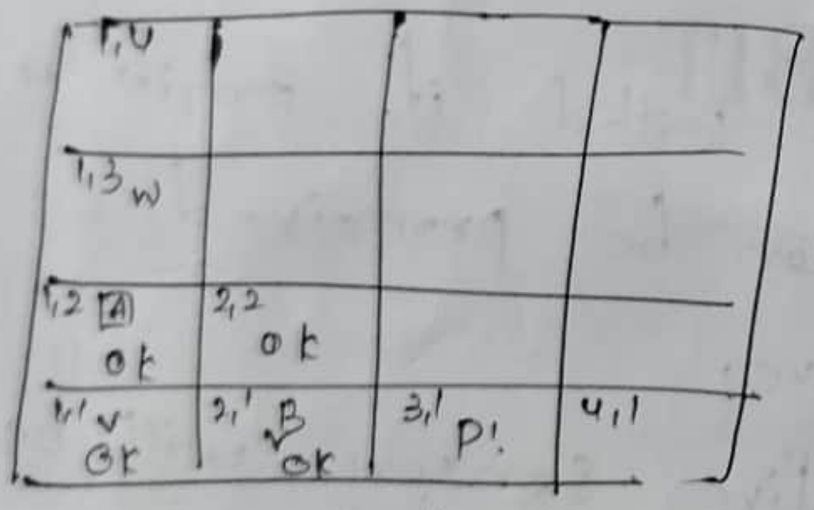


(b)

The first step taken by the agent in the wumpus world.

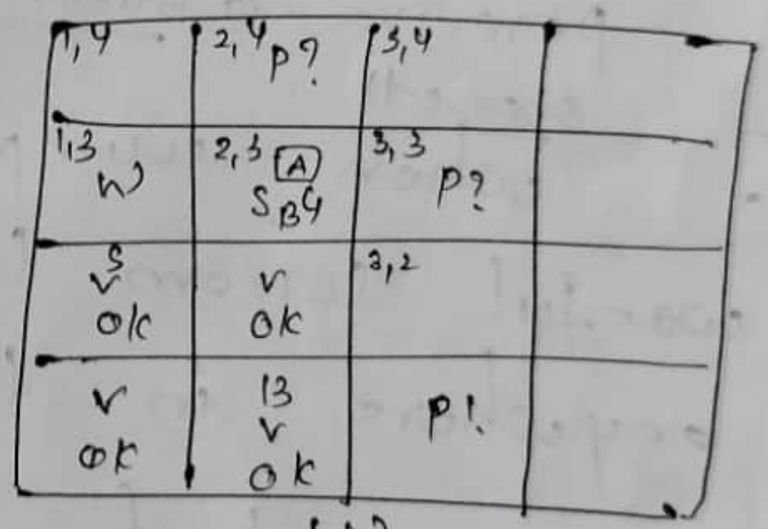
a. The initial situation. after percept & [None, None, None, None, None]

b. After one move, with percept [None, Breeze, None, None, None].



(a)

After the third move with percept [stench, None, None, None, None]



(b)

After fifth move with percept [stench, Breeze, Glitter, None, None].

2.13

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem

Subject :

H.T. No. :
Date :
Signature of Invigilator

- A Horn clause is a disjunction of literals.
ex: The clause $(\neg L_{1,1} \vee \neg Breeze \vee B_{1,1})$ where $L_{1,1}$ is an agent is +ve literal is a Horn clause
ie $(\neg B_{1,1} \vee P_{1,2} \vee P_2)$, is not.

- every Horn clause can be written as an implication of +ve literals whose premise is a conjunction conclusion is a single +ve literal.

- The Horn clause $(\neg L_{1,1} \vee \neg Breeze \vee B_{1,1})$ can be written as implication $(L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$

- A definite clause with no negative literals simply asserts a given proposition sometimes called fact.

- The positive literal is called head and -ve literals from the body of the clause.

EB-SBIT

- prove the KB $(B \vee D) \wedge C$ Based on the given KB.

$[((A \vee B) \Rightarrow C) \wedge (B \vee \neg D), A \wedge D]$

proof:

$$KB = \{\underbrace{((A \vee B) \Rightarrow C) \wedge (B \vee \neg D)}_{KB_1}, \underbrace{A \wedge D}_{KB_2}\}$$

In this premise two KB are given.

considering first KB i.e

S_1: $((A \vee B) \Rightarrow C) \wedge (B \vee \neg D)$

apply AND elimination

S_2: $((A \vee B) \Rightarrow C)$

consider the 2nd KB $A \wedge D$ and apply AND elimination

S_3: $A \wedge D \vdash A$

S_4: A

using a rule called OR Introduction

for S_4 apply OI by using B literal

S_5: $A \vee B$

from statement S_2 and S_5 apply modus ponens $\alpha \rightarrow \beta$, α Result β

S_6: C

Again from the given KB, consider [AND elimination] we get KB, and apply known.

consider

S_7 : B ∨ ¬D

from the second KB_2 we get 'D' By applying AND elimination. (AND) rotrope

S_8 : D

EB-SBIT

(α ∨ ¬B , B we get resolution

a/c to unit resolution

By applying unit resolution on S_6 and S_7

we get.

S_9 : B

By applying OR Introduction (OR) we get

S_{10} : B ∨ D

from S_6 and S_{10} if introducing AND Introduction it will give

S_{11} : (B ∨ D) ∧ C

∴ Hence proved.

Horn clause :. A disjunction of literals where no more than one literal is positive.

Ex: $\neg A \vee \neg B \vee C$ (or)

$(A \wedge B) \rightarrow C$.

Definite clause : It is a type of Horn clause containing exactly one +ve literal and zero or more negative literals.

$A \Rightarrow B \equiv \neg A \vee B$ (or)

$= \neg A \vee B \vee (C \vee \neg C)$

Syntax of propositional logic

logical constants : True / false.

propositional symbols : P, Q, R, ... representing specific fact statements about the world.

- If P is a sentence/statement then P is a term/state
  (if P and Q are statements then they are
  - $P \wedge Q$ conjunction, P and Q are conjuncts
  - $P \vee Q$ disjunction, P and Q are disjuncts
  - $P \Rightarrow Q$ Implication, P is premise/antecedent and Q is consequent / conclusion
  - $P \Leftrightarrow Q$ Biconditional / if and only if equivalence.
  - $\neg P$, negation ($\neg$)

Constants and symbols are atomic, other sentences are complex.